Advanced Learning Algorithms
Coursera Course 2/3 Notes
Nicolas Rebollo Canedo-Arguelles
Started Dec 21, 2025

Table of Contents

1. Week 1 Notes - Neural Networks....3
1.1. Neurons and the Brain....3
1.2. Demand Prediction....3
1.3. Example: Recognizing Images....4
1.4. Neural Network Layer....4
1.5. More Complex Neural Networks....5
1.6. Inference: Making Predictions (Forward Propagation)....6
1.7. Inference in Code....7
1.8. Data in TensorFlow....8
1.9. Building a Neural Network....8
1.10. Forward Propagation in a Single Layer....9
1.11. General Implementation of Forward Propagation....10
1.12. Is There a Path to AGI?....10
1.13. How Neural Networks are Implemented Efficiently....10
1.14. Matrix Multiplication....11
1.15. Matrix Multiplication Rules....12
1.16. Matrix Multiplication Code....12
2. Week 2 Notes - Neural Network Training....13
2.1. TensorFlow Implementation....13
2.2. Training Details....14
2.3. Alternative to the Sigmoid Activation....15
2.4. Choosing Activation Functions....15
2.5. Why Do We Need Activation Functions?....16
2.6. Multiclass....17
2.7. Softmax....17
2.8. Neural Network with Softmax Output....18
2.9. Improved Implementation of Softmax....18
2.10. Classification with Multiple Outputs....19
2.11. Advanced Optimization....20
2.12. Additional Layer Types....21
3. Week 3 Notes - Advice for Applying Machine Learning....21
3.1. Deciding what to try next....21

3.2. Evaluating a model.... 21
3.3. Model selection & training/cross validation/test sets.... 22
3.4. Diagnosing bias and variance.... 22
3.5. Regularization and bias/variance.... 23
3.6. Establishing a baseline level of performance.... 24
3.7. Learning curves.... 24
3.8. Deciding what to try next revisited.... 25
3.9. Bias/variance and neural networks.... 26
3.10. Iterative loop of ML development.... 27
3.11. Error analysis.... 27
3.12. Adding data.... 27
3.13. Transfer learning: using data from a different task.... 28
3.14. Full cycle of machine learning project.... 29
3.15. Fairness, bias, and ethics.... 30
3.16. Error metrics for skewed datasets.... 30
3.17. Trading off precision and recall.... 30
4. Week 4 Notes - Decision Trees.... 31
4.1. Decision tree model.... 31
4.2. Learning process.... 32
4.3. Measuring purity.... 32
4.4. Choosing a split: Information Gain.... 32
4.5. Putting it together.... 33
4.6. Using one-hot encoding of categorical features.... 34
4.7. Continuous valued features.... 34
4.8. Regression trees.... 34
4.9. Using multiple decision trees.... 35
4.10. Sampling with replacement.... 35
4.11. Random forest algorithm.... 36
4.12. XGBoost.... 36
4.13. When to use decision trees.... 37

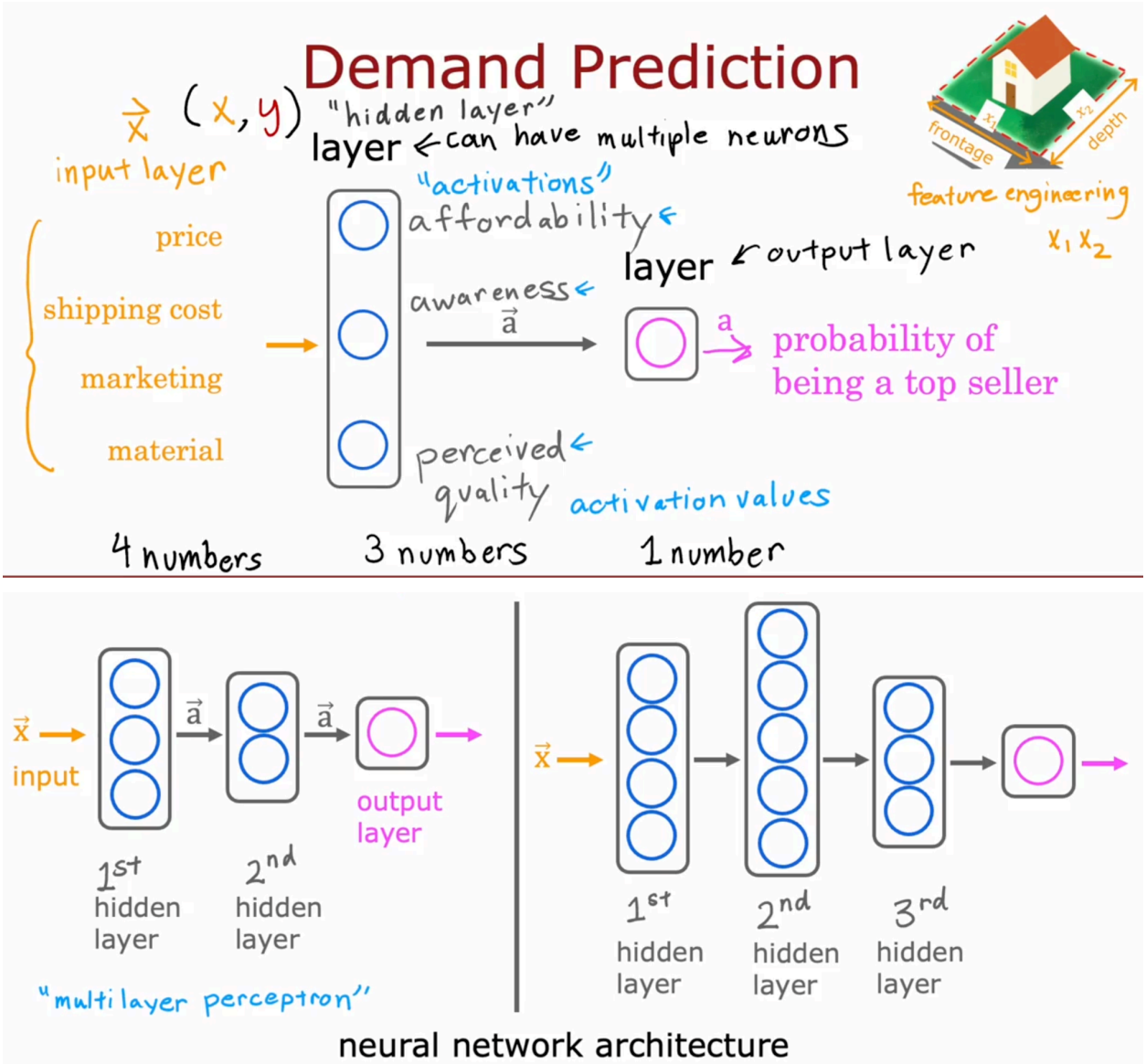
1. Week 1 Notes - Neural Networks

1.1. Neurons and the Brain

- Neural networks were made to mimic how the human brain thinks
- Origin: algorithms that try to mimic the brain. Used in the 1980s and early 1990s
- Huge impact on speech recognition from deep learning. Then grew towards computer vision (images). Then into natural language processing (NLP) or text.
- A neuron has a number of inputs that receives electrical impulses from other neurons and carries out numerical computations and then send its output to other neurons via electrical impulses
- A biological neuron has a cell body (nucleus) which accepts different inputs into the dendrites and sends outputs through the axon
- Artificial neural networks use a simplified mathematical model of a neuron. Takes some inputs, does a computation, and generates an output that can be an input to a second neuron. Rather than building one neuron at a time, you want to simulate many at the same time.

1.2. Demand Prediction

- Want to predict whether a product will be a top seller
- $input = x = price;\ output = a = activation = f(x) = \frac{1}{1+e^{-(wx+b)}}$
- $x\ (price) \rightarrow neuron \rightarrow a\ (probability\ of\ being\ a\ topseller)$
- *Layer*: is a grouping of neurons which take as input the same/similar features and output a few numbers together. Layers can have one or more neurons. The output layer is the layer that creates the final output of the model. The input layer is the inputs to the first layer of neurons in the model. The layer in the middle is the "hidden layer", where the input layer provides its inputs and this layer provides the outputs to the output layer.
- *Activations:* outputs from layers
- Each neuron in each layer has access to every feature from the previous layer (to avoid hand picking features manually). It will figure out which features are most relevant.
- $\vec{x} \rightarrow \vec{a} \rightarrow a$ (input vector, activations vector, activation)
- The neural network figures out the features to use in its hidden layer, which is what makes it so powerful.
- *Neural Network Architecture:* Can have multiple hidden layers. Need to decide how many hidden layers and how many neurons in each hidden layer to have. A network with multiple hidden layers is also called a "multilayer perceptron"



1.3. Example: Recognizing Images

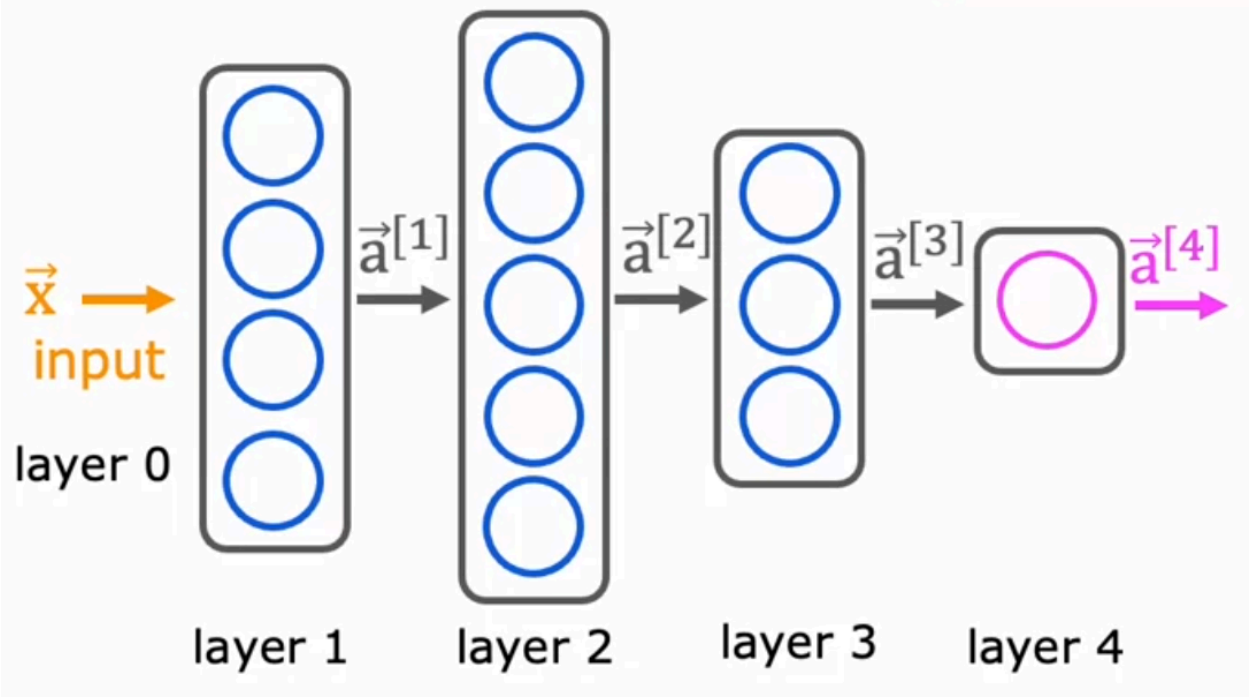
- Could have a matrix of 1000 x 1000 pixels (intensity values)
- Unravel into 1 million long vector and feed into a neural network with many hidden layers
- Output layer provides as output the probability of being person 'XYZ'

1.4. Neural Network Layer

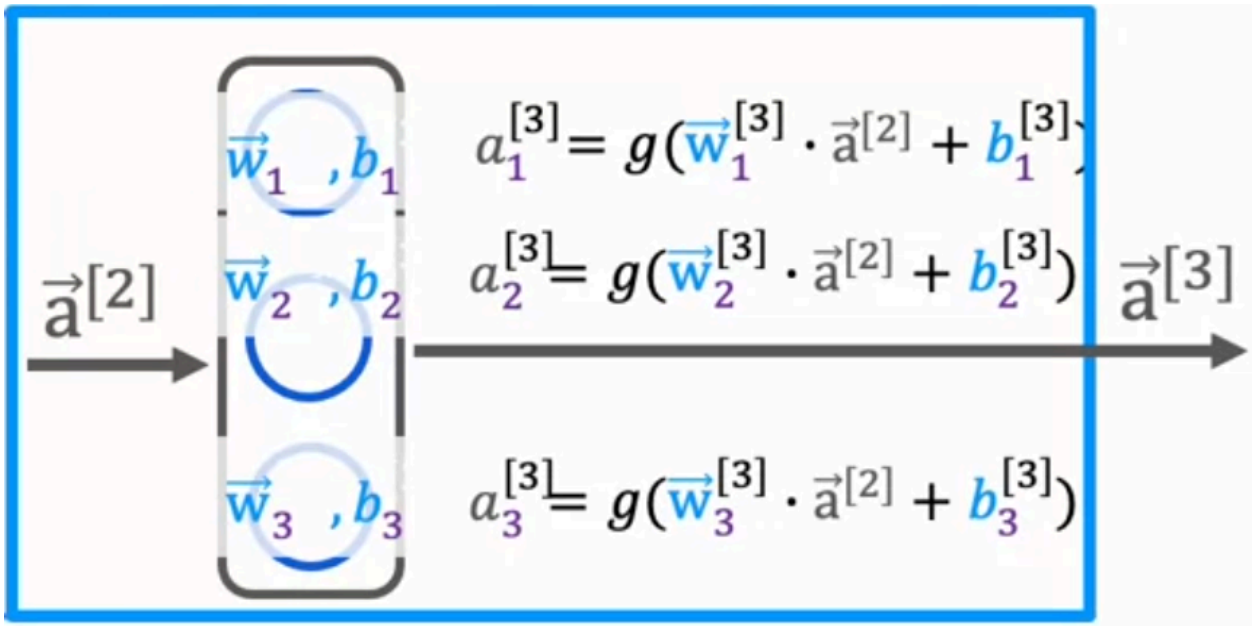
- Layer of neurons
- Each neuron takes the inputs and does a logistic regression
    - Each neuron has its own parameters ($\vec{w}_i$, $\vec{b}_i$)

    - Activation value of each neuron is the output, e.g., $a_1 = g(\vec{w}_1 \cdot \vec{x} + b_1)$
- By convention, the first layer is layer 1, the second one is layer 2
    - The input layer is sometimes called layer 0
    - Notation for layer numbering: $\vec{a}^{[1]}$ (output of layer 1 hidden layer)
- Vector of activation values from layer 1 becomes the input to layer 2 (output layer in our case)
    - Now activation of layer 2 becomes $a_2 = g(\vec{w}_1 \cdot \vec{a}^{[1]} + b_1) = a^{[2]}$ (final output of NN)
- Final optional step is to predict a category 1 or 0 (yes/no)
    - Is $a^{[2]} \geq 0.5$? Yes: $\hat{y} = 1$; No: $\hat{y} = 0$

1.5. More Complex Neural Networks



- 4 layers (not counting the input layer). Layers 1, 2, 3 are hidden layers, layer 4 is output layer, layer 0 is the input layer. Layer count includes hidden and output layers
- Layer 3 computations:

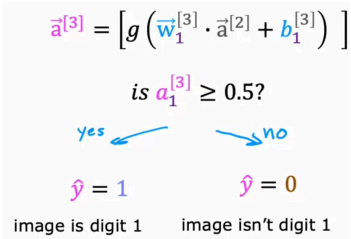


- $\vec{a}^{[3]} = [a_1^{[3]}, a_2^{[3]}, a_3^{[3]}]$ → output of layer 3
- For arbitrary layer, l, and arbitrary unit (neuron), j:
  - $a_j^{[l]} = g(\vec{w}_j^{[l]} \cdot \vec{a}^{[l-1]} + b_j^{[l]})$
    - g is the *sigmoid* or *activation* function
- Input vector: $\vec{x} = \vec{a}^{[0]}$

1.6. Inference: Making Predictions (Forward Propagation)

- Forward propagation algorithm to let NN make inferences/predictions
- Example: binary classification of digit recognition (0 or 1)
  - 2 hidden layers
    - First has 25 units, second has 15 units, output gives probability of being a handwritten '1'

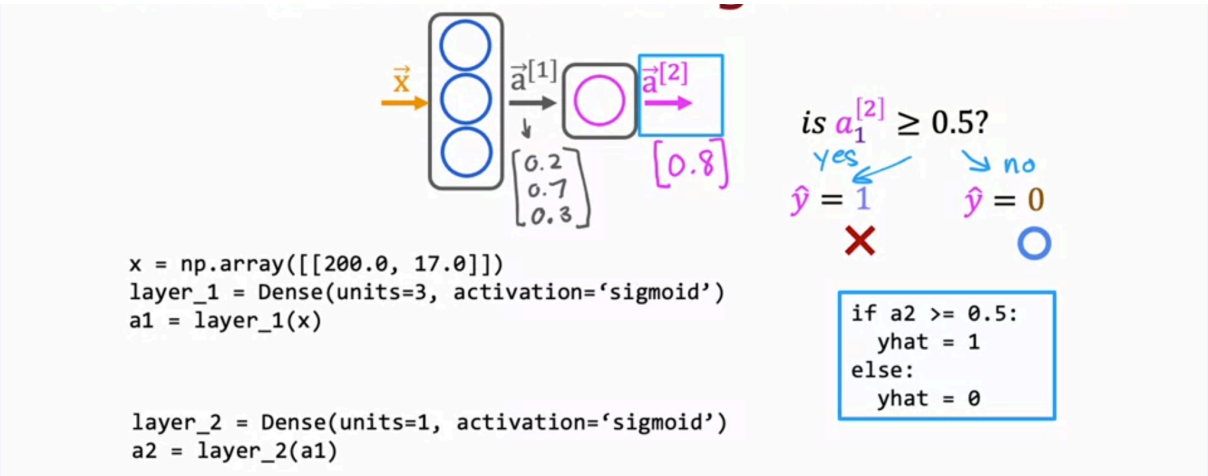
$$\vec{a}^{[1]} = \begin{bmatrix} g(\vec{w}_1^{[1]} \cdot \vec{x} + b_1^{[1]}) \\ \vdots \\ g(\vec{w}_{25}^{[1]} \cdot \vec{x} + b_{25}^{[1]}) \end{bmatrix} \quad \vec{a}^{[2]} = \begin{bmatrix} g(\vec{w}_1^{[2]} \cdot \vec{a}^{[1]} + b_1^{[2]}) \\ \vdots \\ g(\vec{w}_{15}^{[2]} \cdot \vec{a}^{[1]} + b_{15}^{[2]}) \end{bmatrix}$$



- Note that $\vec{a}^{[3]} = f(x)$ is the output of the neural network
- Forward propagation since you are propagating the activations of the neurons from input to output. This is in contrast to *back propagation* which is used for learning
- Architecture where number of units decreases as you go through layers is common

1.7. Inference in Code

- TensorFlow leading framework for neural network algorithms. PyTorch also popular



- Layer 1:

```
x = np.array([[200.0, 17.0]])
layer_1 = Dense(units=3, activation='sigmoid')
a1 = layer_1(x)
```

- Layer 2:

```
layer_2 = Dense(units=1, activation='sigmoid')
a2 = layer_2(a1)
```

- Threshold:

```
if a2 >= 0.5:
     yhat = 1
else:
     yhat = 0
```

- For digit classification example:

```
x = np.array([[0.0, ..., 245, ..., 240, ..., 0]])
layer_1 = Dense(units=25, activation='sigmoid')
a1 = layer_1(x)
layer_2 = Dense(units=15, activation='sigmoid')
a2 = layer_2(a1)
layer_3 = Dense(units=1, activation='sigmoid')
a3 = layer_2(a1)
```

- Threshold:

```
if a3 >= 0.5:
      yhat = 1
else:
      yhat = 0
```

1.8. Data in TensorFlow

```
x = np.array([[200.0, 17.0]])
```

- Why a double square bracket? Matrix or 2D array, 1 x 2 in this case (row vector)
- n x m matrix means n rows, m columns:

```
x = np.array([[1,2,3],
              [4,5,6]])
```

- Use matrices in TensorFlow (convention). TF designed to handle large datasets, matrices are more computationally efficient with matrices
- When you do `a1 = layer_1(x)`, that returns a *tensor* (data type for efficient matrix computations). It is essentially a matrix (for the purpose of this course)
- So, you can do `a1.numpy()` to get back to a numpy array

1.9. Building a Neural Network

```
layer_1 = Dense(units=3, activation='sigmoid')
layer_2 = Dense(units=1, activation='sigmoid')
model = Sequential([layer_1, layer_2])

x = np.array([[200.0, 17.0],
              [120.0, 5.0],
              [425.0, 20.0],
              [212.0, 18.0]])
y = np.array([1,0,0,1])

model.compile(...)         # more about this next week
model.fit(x,y)             # train on data x and y

model.predict(x_new)  # forward propagation/inference
```

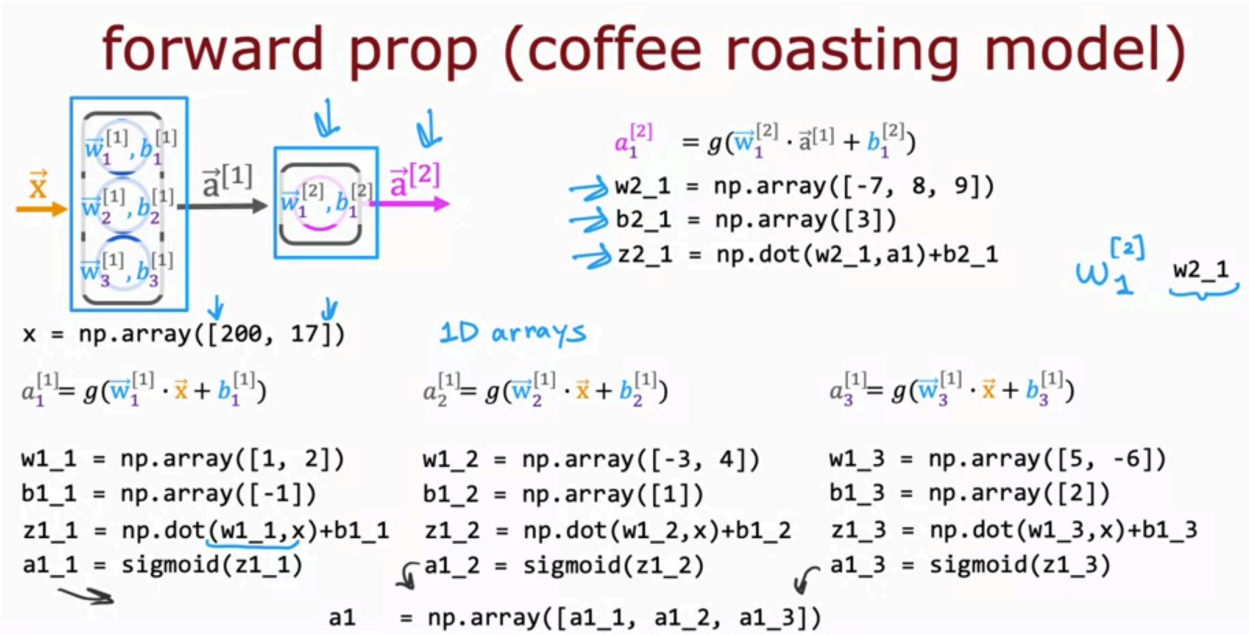
- Can simplify top 3 lines as:

```
model = Sequential([
      Dense(units=3, activation='sigmoid'),
      Dense(units=1, activation='sigmoid')
])
```

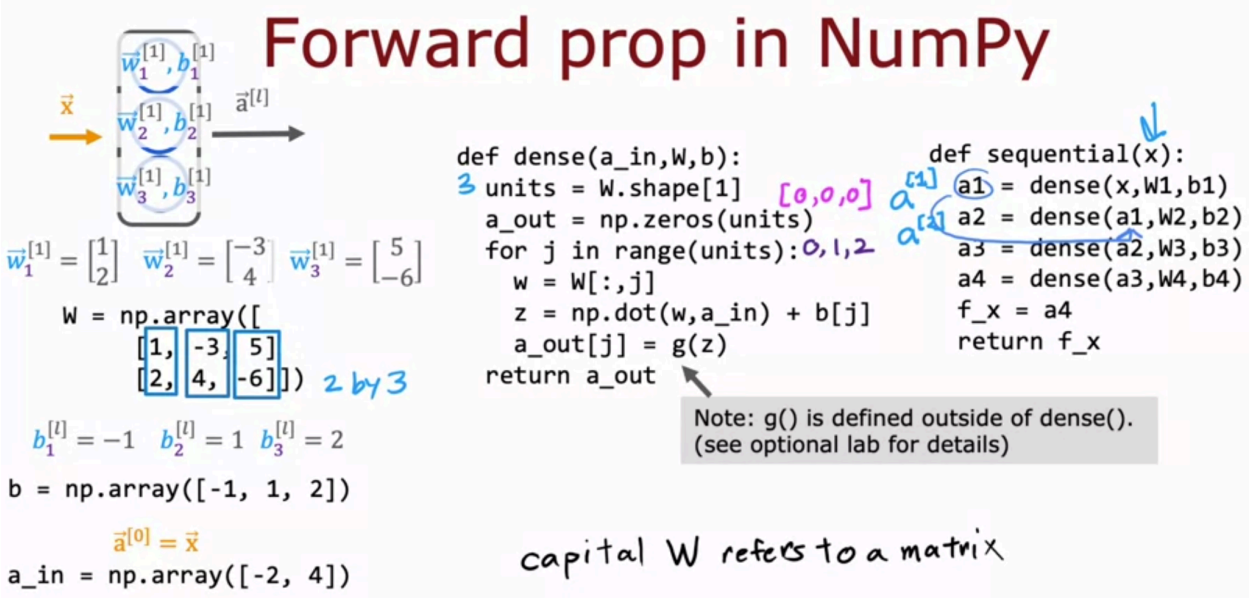
- For digit classification example:

```
model = Sequential([
      Dense(units=25, activation='sigmoid'),
      Dense(units=15, activation='sigmoid'),
      Dense(units=1, activation='sigmoid')
])

model.compile(...)

x = np.array([[0..., 245, ..., 17],
              [0..., 200, ..., 184])
y = np.array([1,0])

model.fit(x,y)
model.predict(x_new)
```

1.10. Forward Propagation in a Single Layer



1.11. General Implementation of Forward Propagation

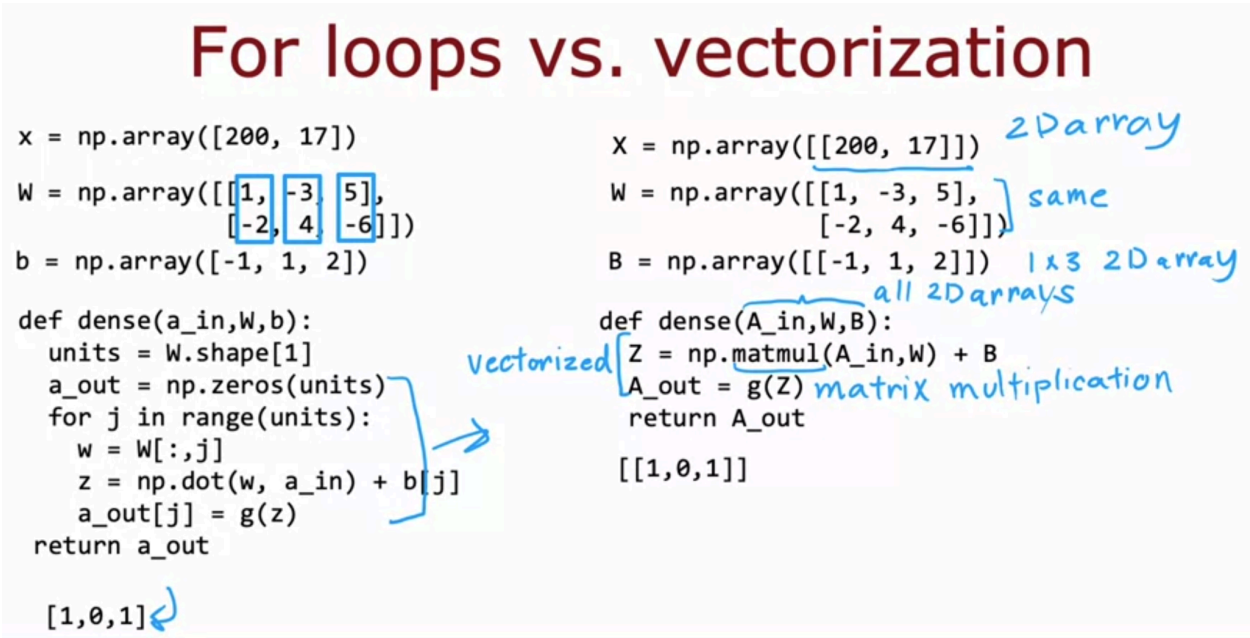


1.12. Is There a Path to AGI?

- AI includes:
  - Artificial Narrow Intelligence (ANI): does one narrow task (smart speaker, self-driving car, web search, etc)
  - Artificial General Intelligence (AGI): AI systems that can do anything a human can do (little progress made)
- We have (almost) no idea how the brain works – simulating the brain in a computer is far from what we have
- "One learning algorithm" hypothesis:
  - Same piece of biological brain tissue can do a wide range of tasks
  - Auditory cortex *learns to see* if the auditory pathway is cut
  - Somatosensory cortex *learns to see* if the connection is cut for touch
  - Sensor representations in the brain (e.g., seeing with your tongue, human echolocation or sonar, haptic belt for direction sense, implanting a 3rd eye)

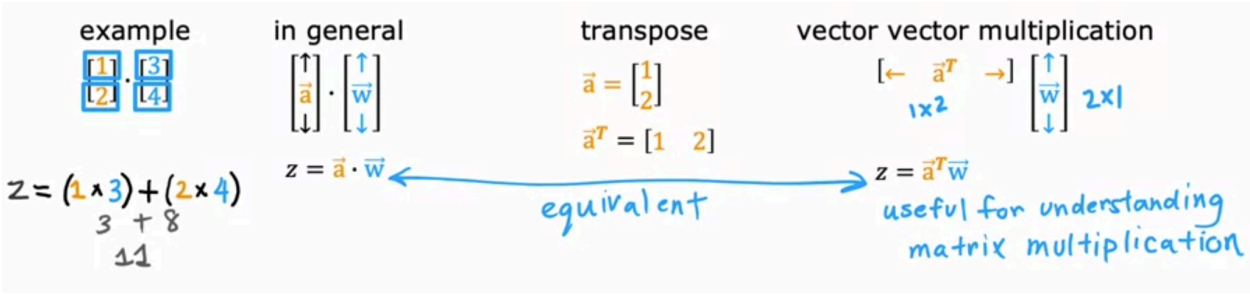
1.13. How Neural Networks are Implemented Efficiently

- NNs can be vectorized with matrix multiplication
- Forward propagation for loop vs. vectorization:

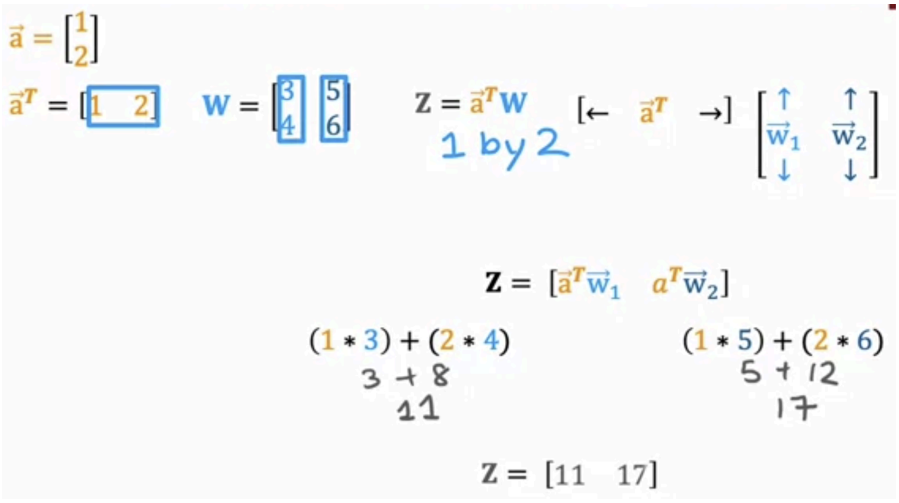


1.14. Matrix Multiplication

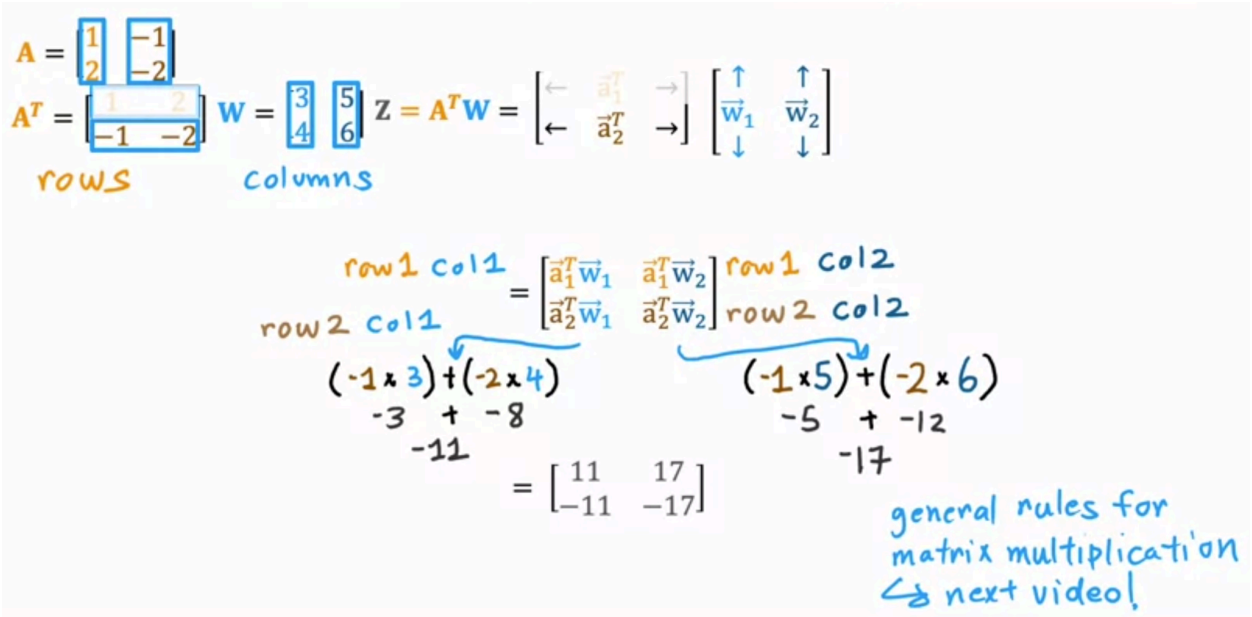
- Dot products:



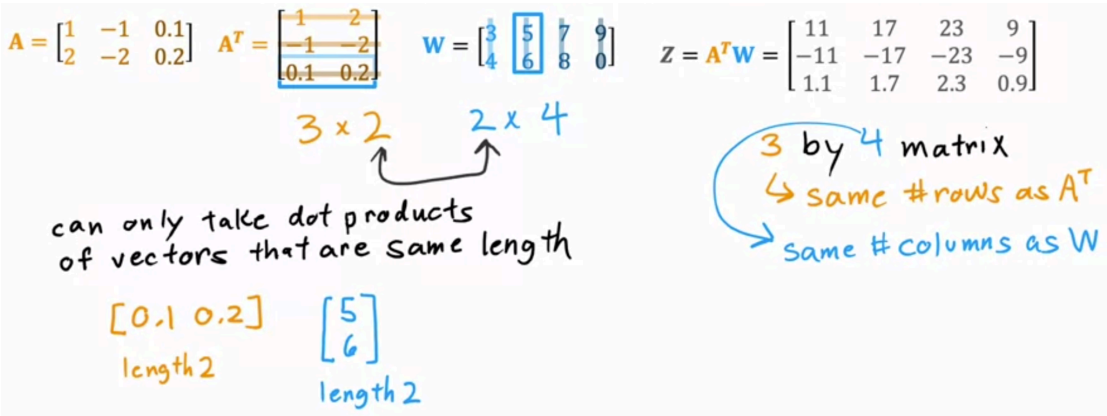
- Vector-matrix multiplication



- Matrix-matrix multiplication

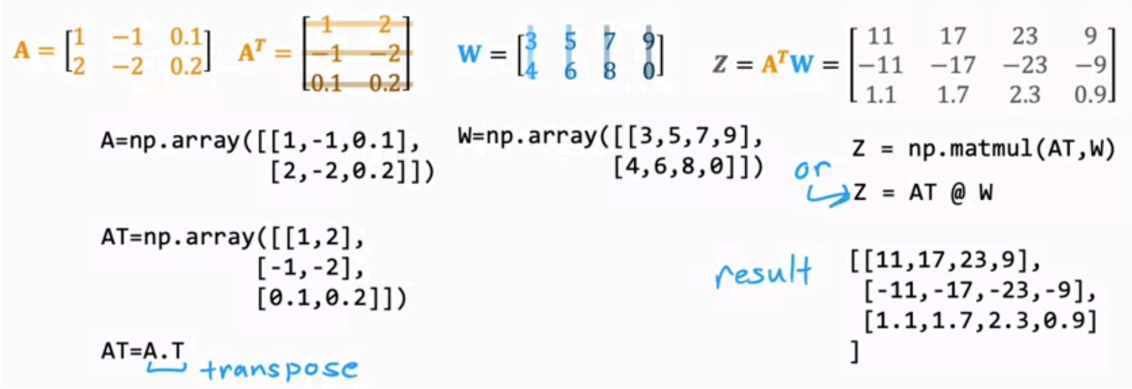


1.15. Matrix Multiplication Rules

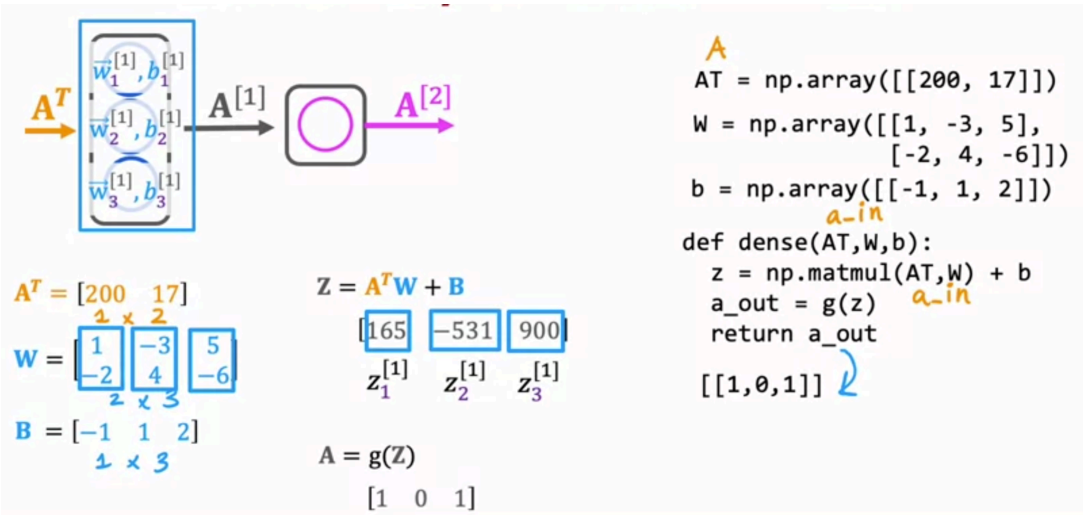


1.16. Matrix Multiplication Code

- Matrix multiplication in Numpy:

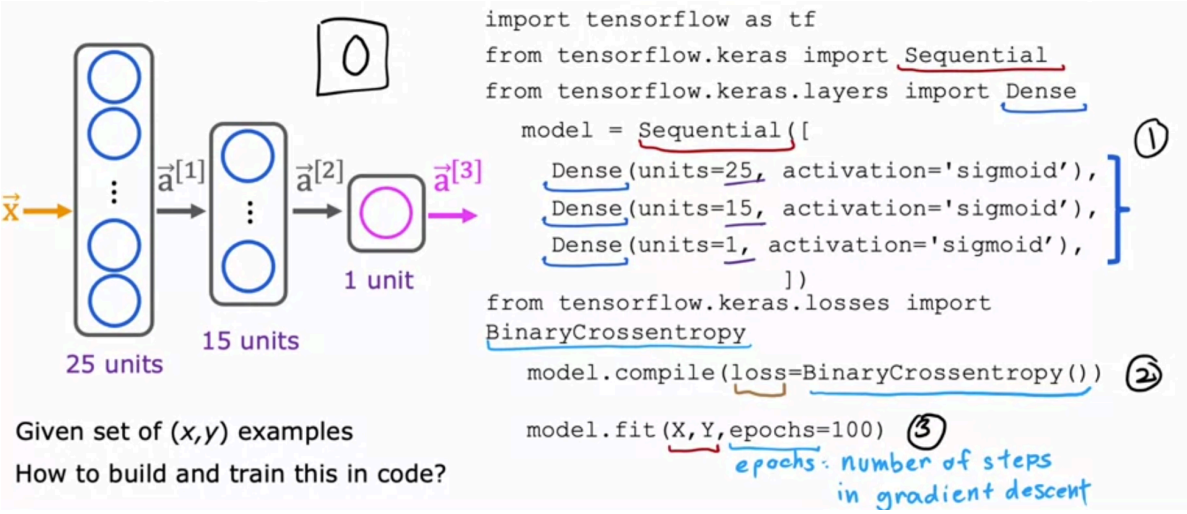


- Dense layer vectorized:



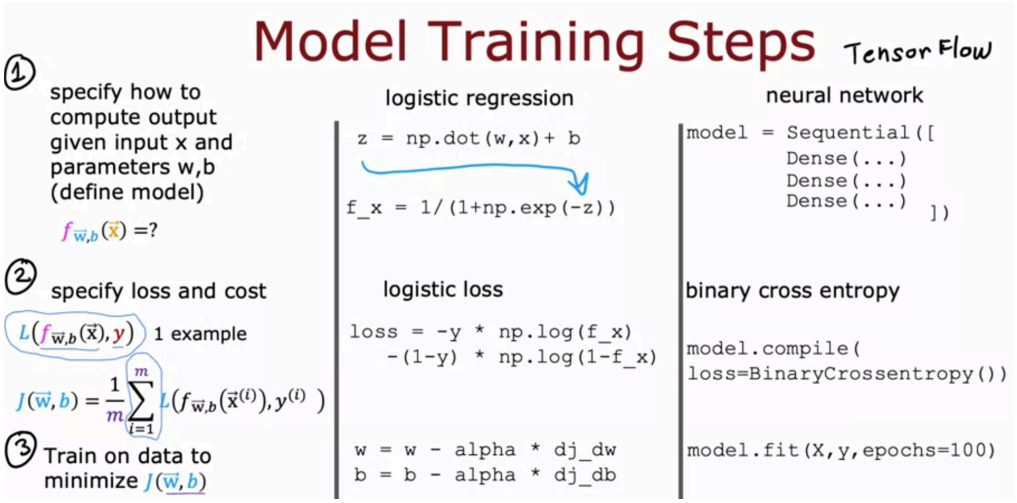
2. Week 2 Notes - Neural Network Training

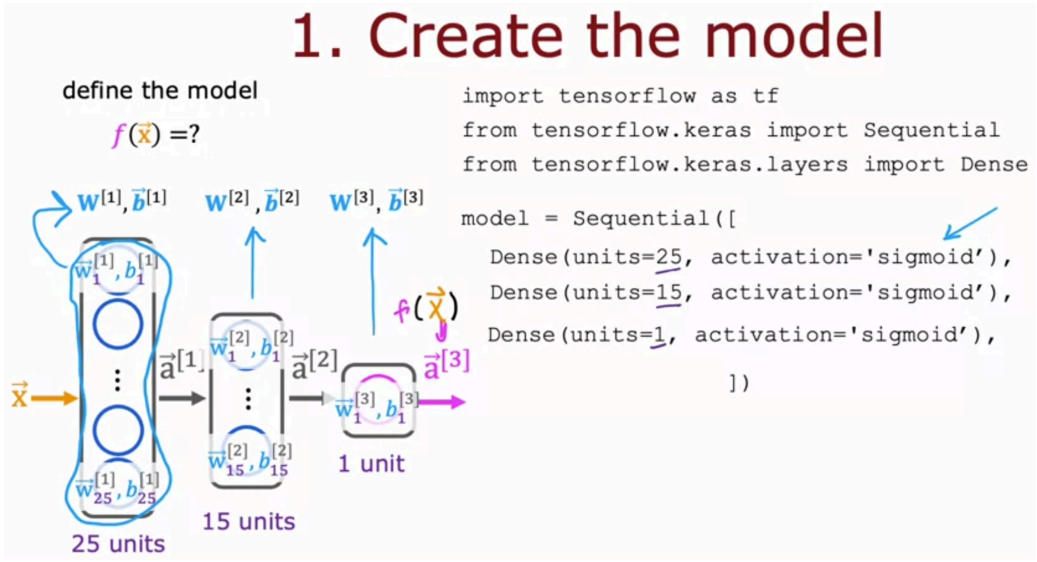
2.1. TensorFlow Implementation

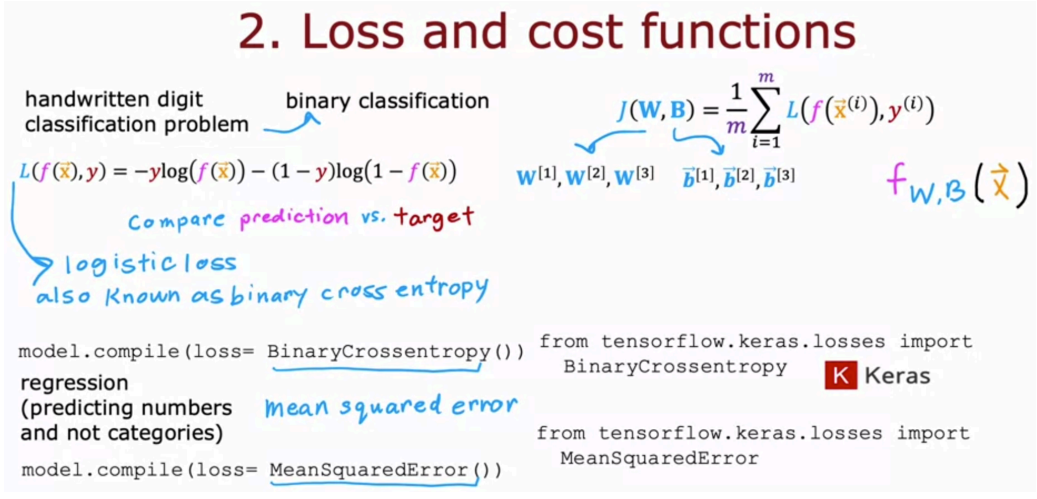


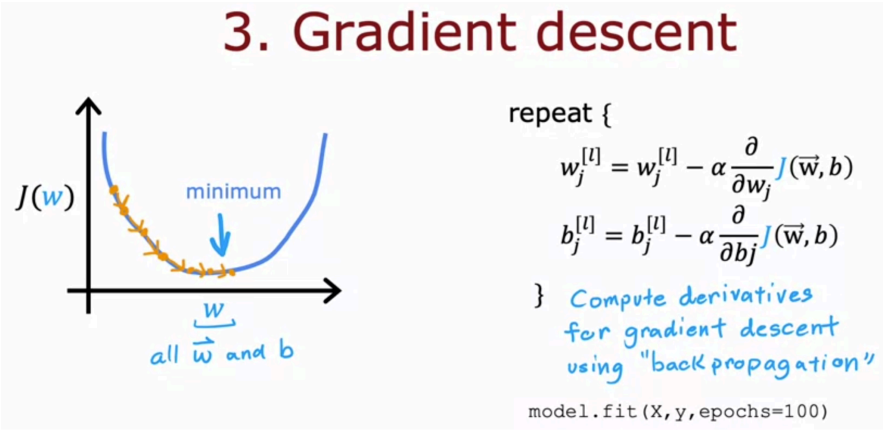
- First build sequential model with dense hidden layers, then compile model with a specified loss function (BinaryCrossentropy), and finally fit or train the model on your data with a specified number of epochs (steps in gradient descent)

2.2. Training Details

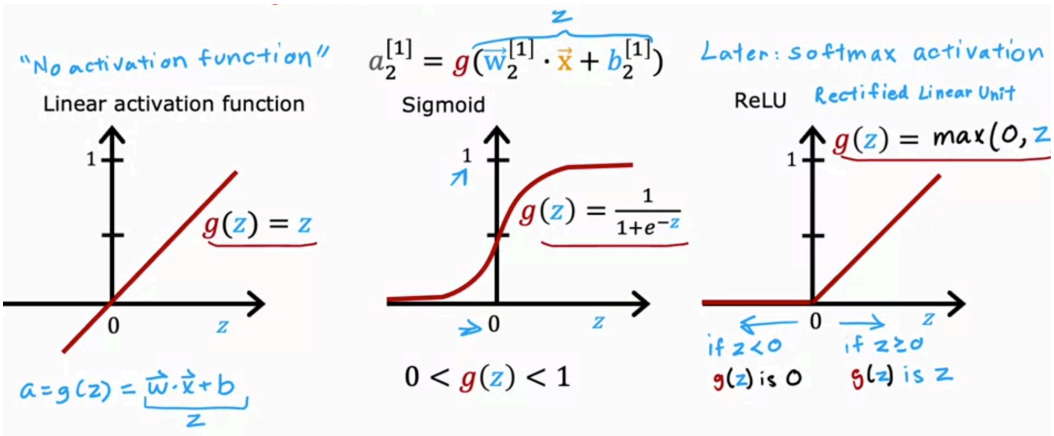








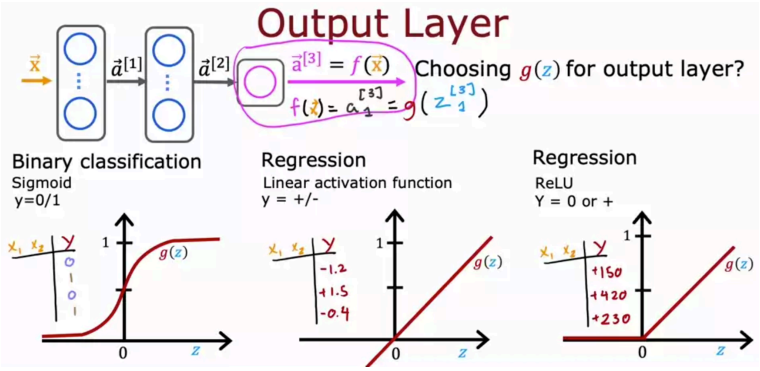
2.3. Alternative to the Sigmoid Activation

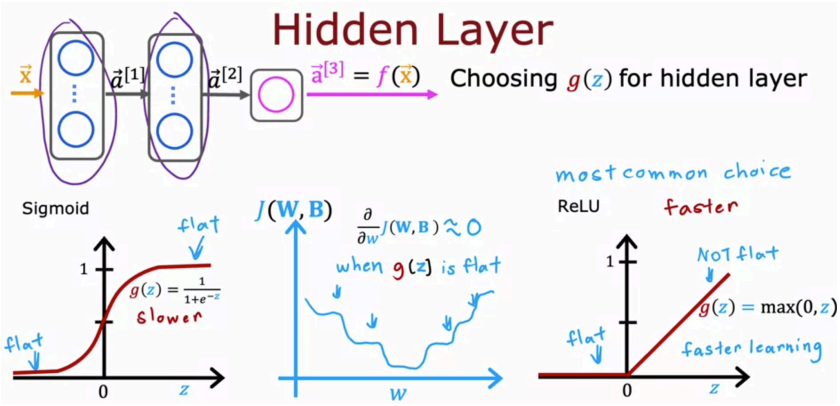


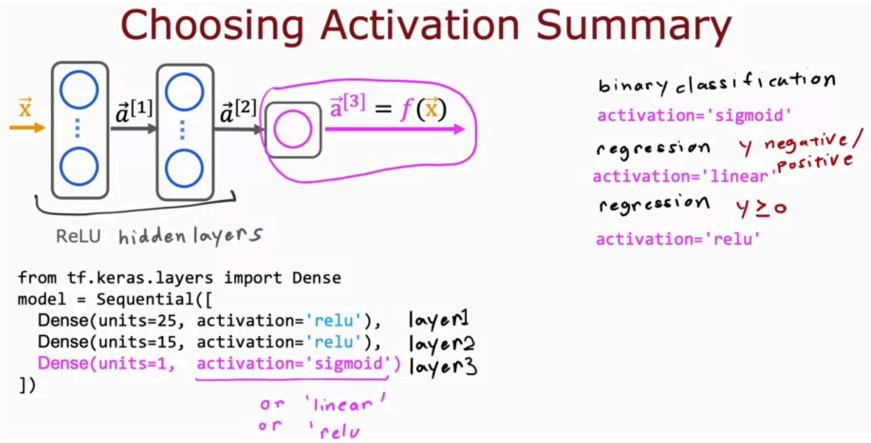
- Most common activation function options include:
  - Linear activation function (no activation function, $g(z) = z$)
  - Sigmoid ($g(z) = \frac{1}{1+e^{-z}}$; $0 < g(z) < 1$)
  - ReLU, rectified linear unit ($g(z) = max(0, z)$; $g(z)$ *is* 0 *when* $z < 0$)
  - Also softmax, mentioned later

2.4. Choosing Activation Functions

- Can choose different activation functions for different neurons in NN

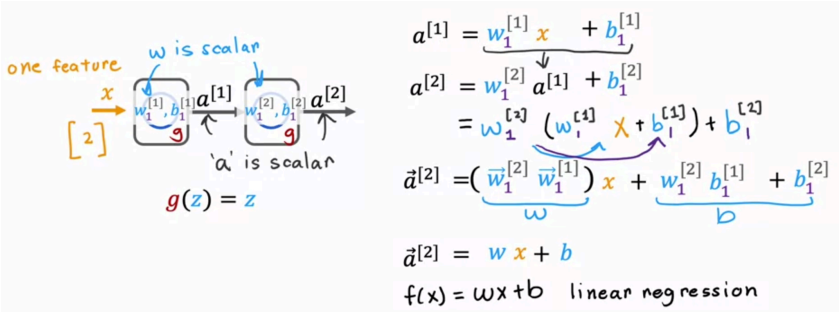


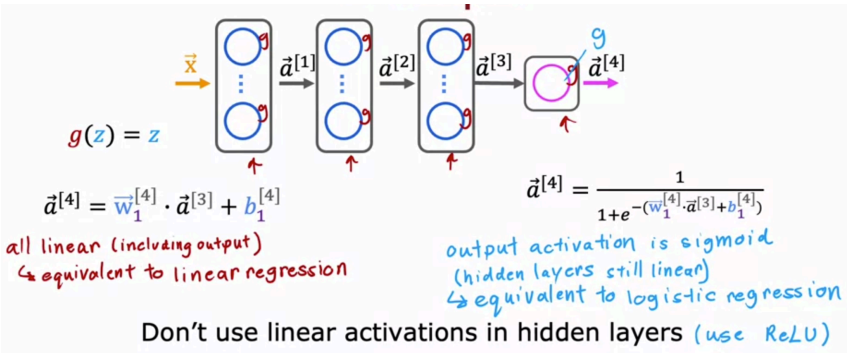




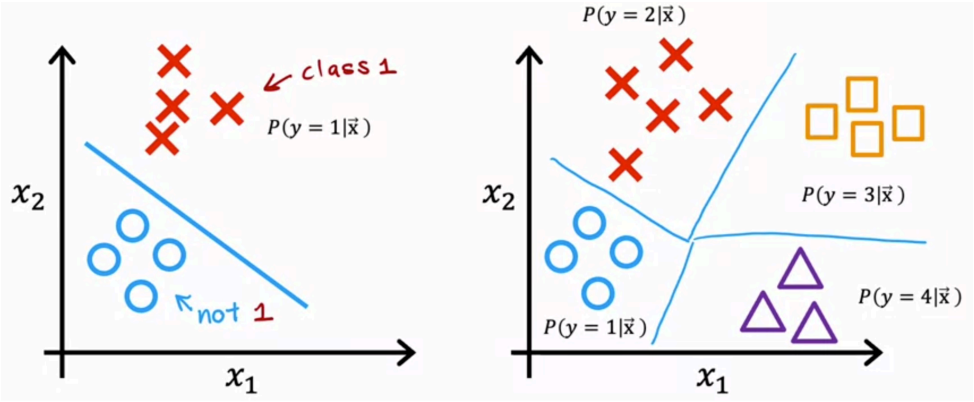
2.5. Why Do We Need Activation Functions?

- Using linear activation functions at all neurons essentially leads to a linear regression, not a neural network





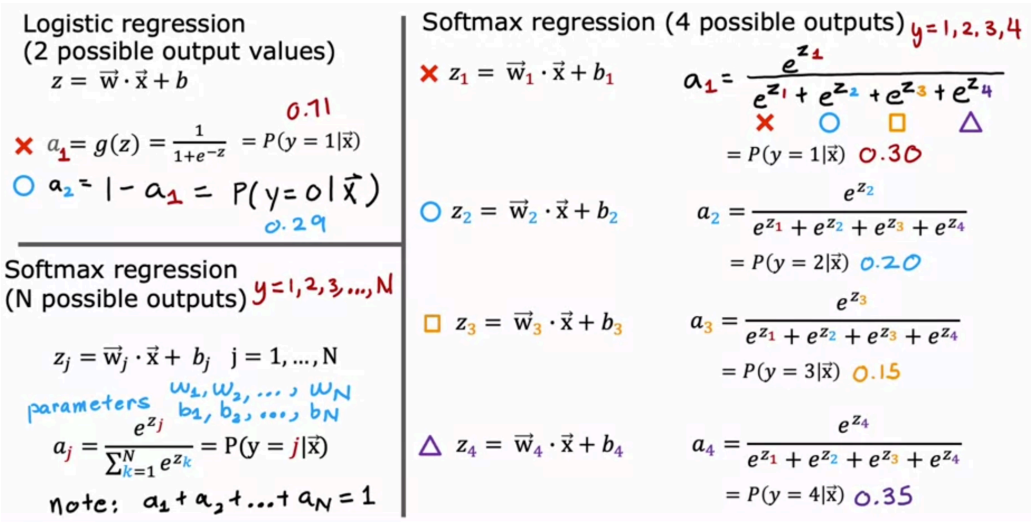
2.6. Multiclass

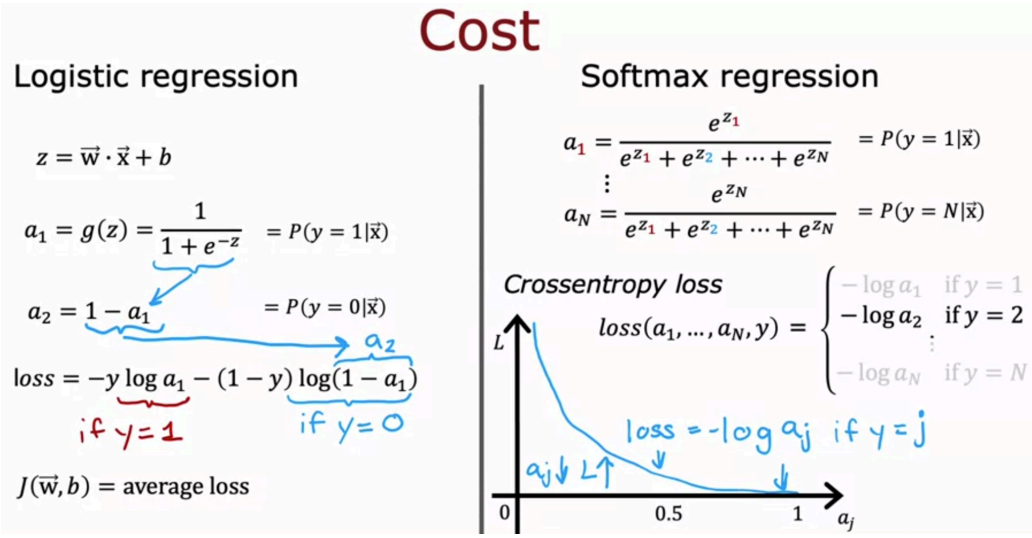


- Many possible classes during classification (more than 2, so not binary classification)

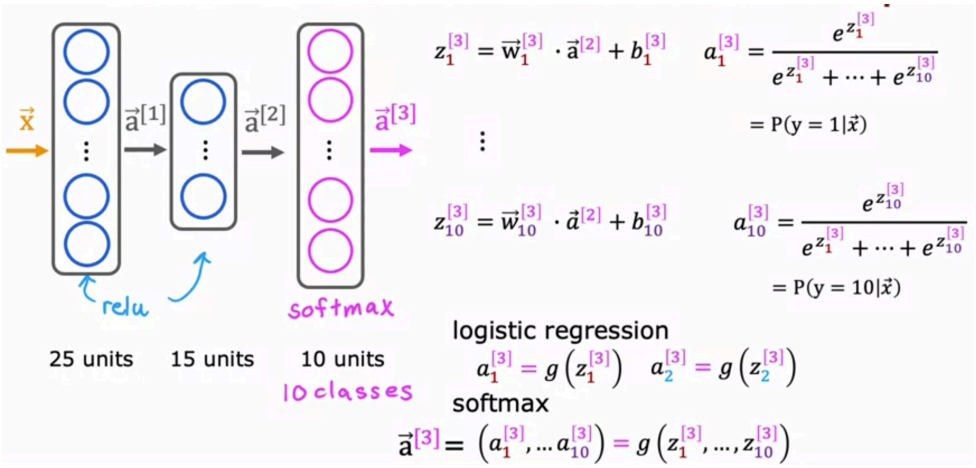
2.7. Softmax

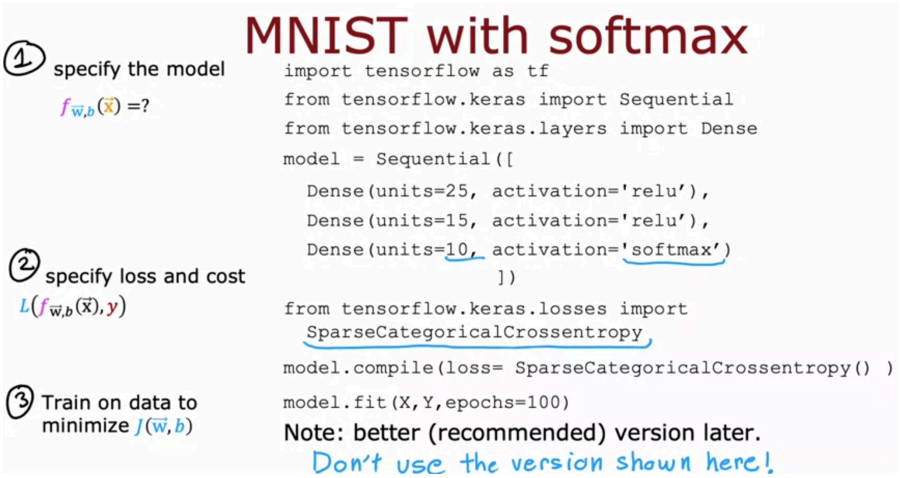
- Generalization of logistic regression to multiclass classification context





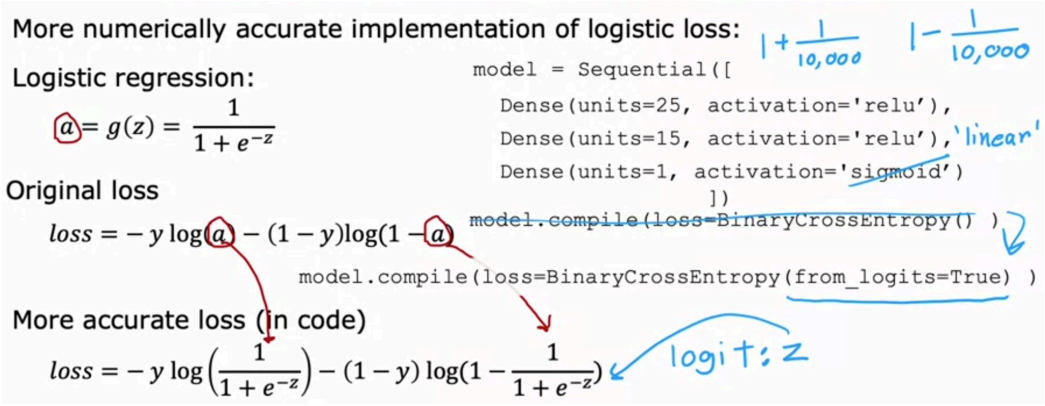
2.8. Neural Network with Softmax Output





2.9. Improved Implementation of Softmax

- Numerical roundoff errors:
  - Option 1: $x = \frac{2}{10{,}000}$
  - Option 2: $x = \left(1 + \frac{1}{10{,}000}\right) - \left(1 - \frac{1}{10{,}000}\right)$
  - Option 1 results in less numerical roundoff error due to less memory use



More numerically accurate implementation of softmax

Softmax regression

$$(a_1, \ldots, a_{10}) = g(z_1, \ldots, z_{10})$$

$$\text{Loss} = L(\vec{a}, y) = \begin{cases} -\log a_1 & \text{if } y = 1 \\ \vdots & \\ -\log a_{10} & \text{if } y = 10 \end{cases}$$

```
model = Sequential([
  Dense(units=25, activation='relu'),
  Dense(units=15, activation='relu'),
  Dense(units=10, activation='softmax')
                  ])
```

'linear'

```
model.compile(loss=SparseCategoricalCrossEntropy() )
```

More Accurate

$$L(\vec{a}, y) = \begin{cases} -\log \frac{e^{z_1}}{e^{z_1} + \cdots + e^{z_{10}}} & \text{if } y = 1 \\ \vdots & \\ -\log \frac{e^{z_{10}}}{e^{z_1} + \cdots + e^{z_{10}}} & \text{if } y = 10 \end{cases}$$

```
model.compile(loss=SparseCategoricalCrossEntropy(from_logits=True))
```

logistic regression (more numerically accurate)

model

```
model = Sequential([
  Dense(units=25, activation='sigmoid'),
  Dense(units=15, activation='sigmoid'),
  Dense(units=1, activation='linear')
                  ])
from tensorflow.keras.losses import
  BinaryCrossentropy
```

loss

```
model.compile(..., BinaryCrossentropy(from_logits=True))
model.fit(X,Y,epochs=100)
```

fit

```
logit = model(X)
```

z

predict

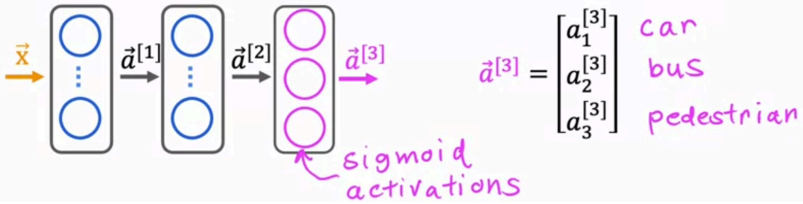
```
f_x = tf.nn.sigmoid(logit)
```

2.10. Classification with Multiple Outputs

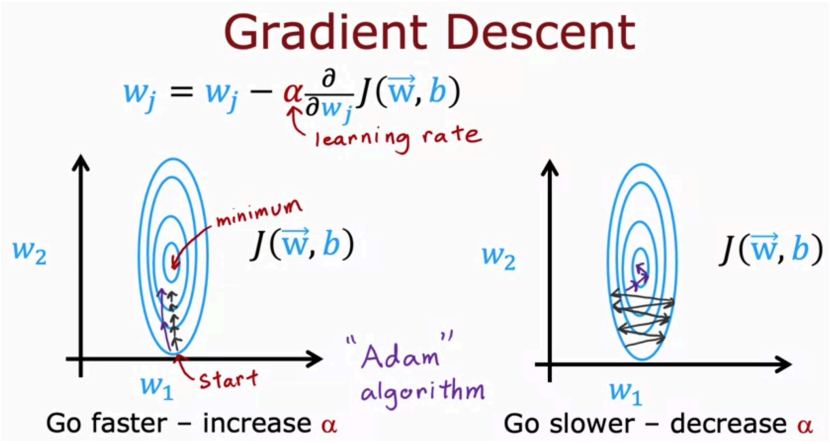
Is there a car? yes
Is there a bus? no
Is there a pedestrian yes

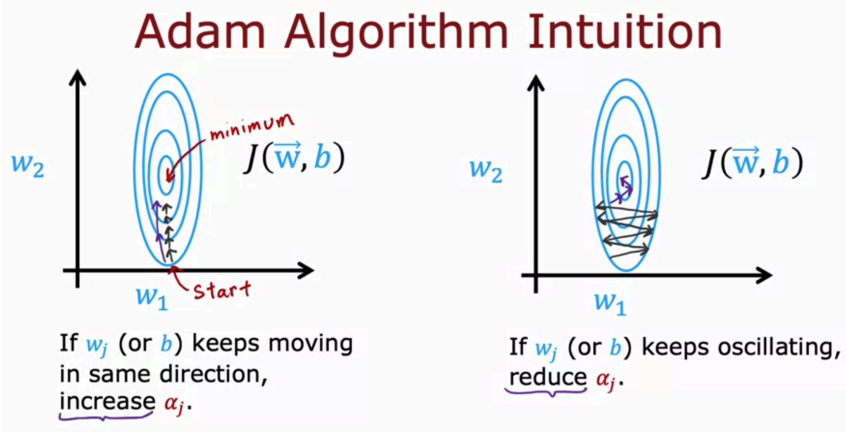
$y = \begin{bmatrix} 1 \\ 0 \\ 1 \end{bmatrix}$ $y = \begin{bmatrix} 0 \\ 0 \\ 1 \end{bmatrix}$ $y = \begin{bmatrix} 1 \\ 1 \\ 0 \end{bmatrix}$

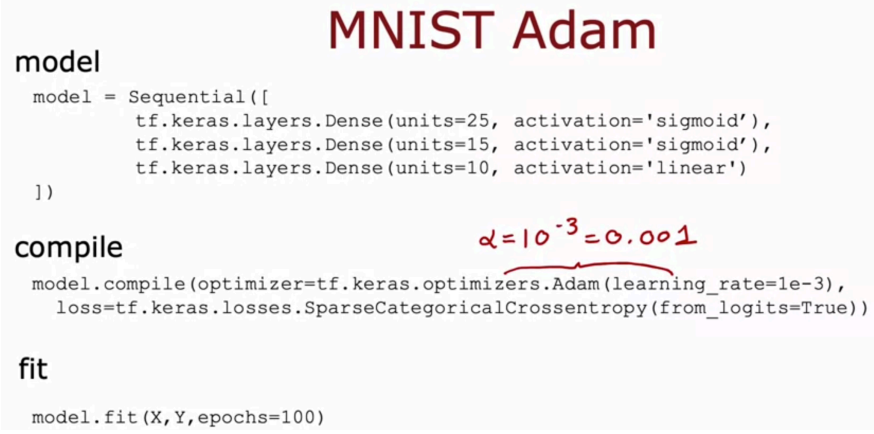
Alternatively, train one neural network with three outputs



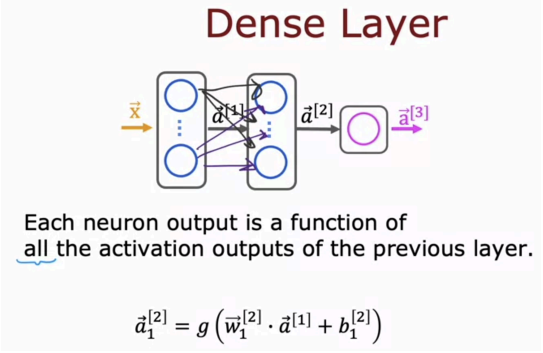
2.11. Advanced Optimization

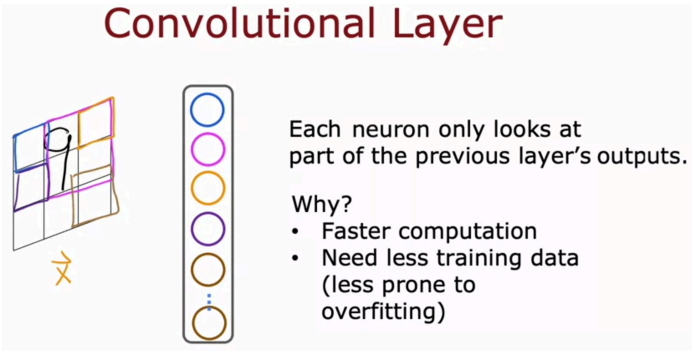


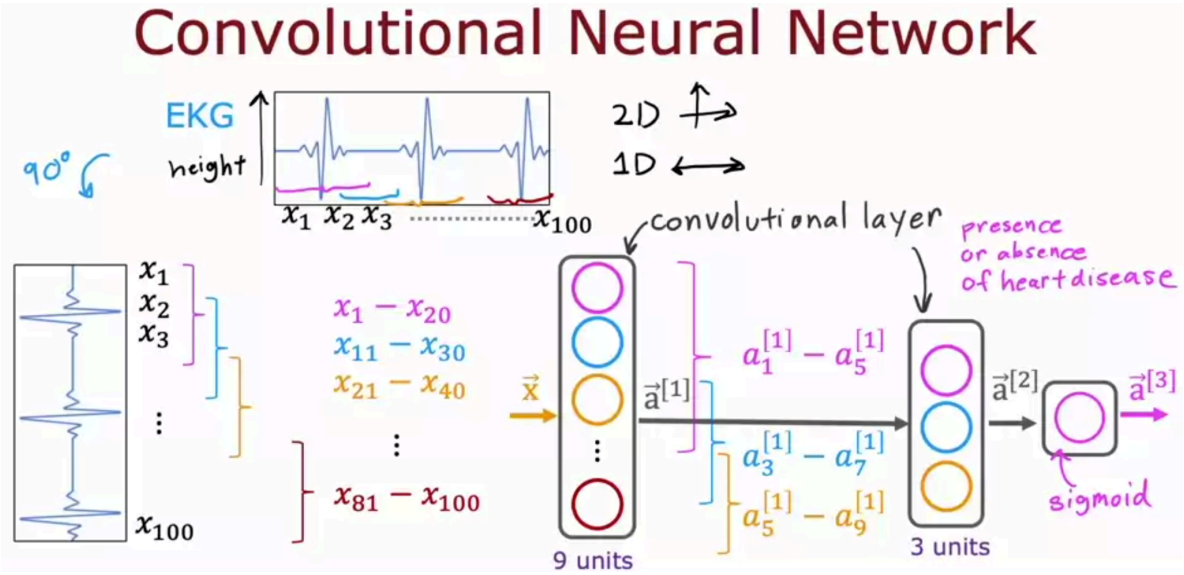




2.12. Additional Layer Types







3. Week 3 Notes - Advice for Applying Machine Learning

3.1. Deciding what to try next

- Efficiency depends how well you make decisions over the machine learning project
- Diagnostic: a test that you run to gain insight into what is/isn't working with a learning algorithm, to gain guidance into improving its performance. They can take time to implement, but running them can be a very good use of your time

3.2. Evaluating a model

- Rather than taking all of your data to train your model, split dataset into two subsets (e.g., 70% training set, 30% test set)
- Train on training set, and evaluate performance on the test set
- For regularized linear regression with squared error cost:

$$J(\vec{w}, b) = \left[\frac{1}{2m_{train}} \sum_{i=1}^{m_{train}} \left(f_{\vec{w},b}(\vec{x}^{(i)}) - y^{(i)}\right)^2 + \frac{\lambda}{2m_{train}} \sum_{j=1}^{n} w_j^2\right]$$

- Then, compute test error:

$$J_{test}(\vec{w}, b) = \frac{1}{2m_{test}}\left[\sum_{i=1}^{m_{test}} (f_{\vec{w},b}(\vec{x}_{test}^{(i)}) - y_{test}^{(i)})^2\right]$$

- Compute training error:

$$J_{train}(\vec{w}, b) = \frac{1}{2m_{train}}\left[\sum_{i=1}^{m_{train}} (f_{\vec{w},b}(\vec{x}_{train}^{(i)}) - y_{train}^{(i)})^2\right]$$

- Fraction of the test and training sets that the algorithm has misclassified:

$$\hat{y} = 1 \; if \; f_{\vec{w},b}(\vec{x}^{(i)}) \geq 0.5; \; 0 \; otherwise$$

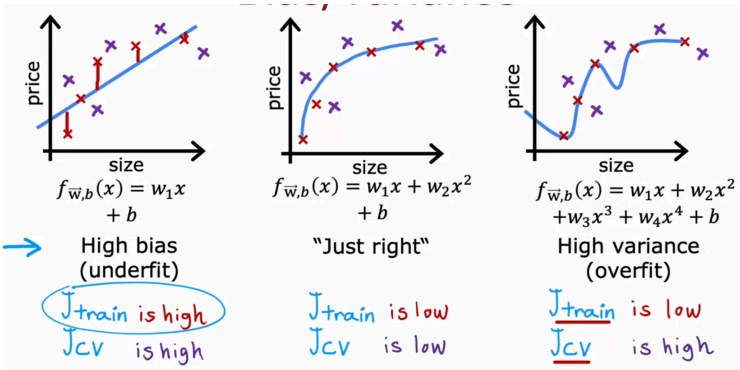
- Then count $\hat{y} \neq y$. J_{test} is the fraction of the test that has been misclassified, similar for the training set.

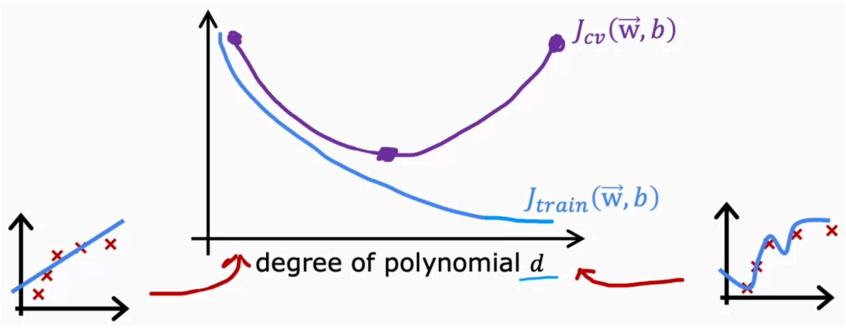
3.3. Model selection & training/cross validation/test sets

- Once parameters are fit to the training set, the training error will likely be lower than the generalization error
- J_{test} a better indicator of generalization error
- Could try to see which model gives the lowest J_{test} value. However, this is likely to be an optimistic estimate of generalization error because an extra parameter (d, degree of polynomial) was chosen using the test set
- Modify procedure to split data into 3 sets (training 60%, cross validation 20%, testing 20%)
- Choose best model using cross validation (cv) set and then use test set to estimate generalization error of the chosen model

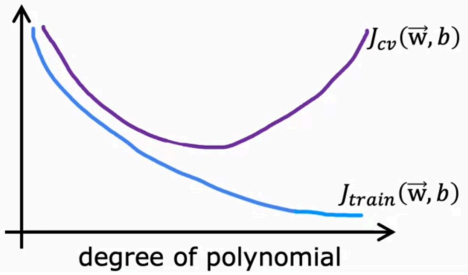
3.4. Diagnosing bias and variance

- How to decide what to do next to improve performance? Look at bias and variance for good guidance into what to try next





How do you tell if your algorithm has a bias or variance problem?



High bias (underfit)
→ J_{train} will be high
($J_{train} \approx J_{cv}$)

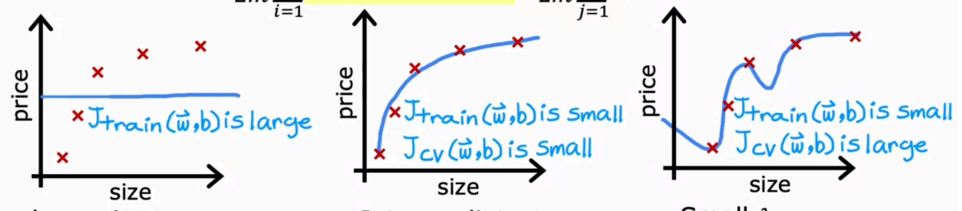
High variance (overfit)
→ $J_{cv} \gg J_{train}$
(J_{train} may be low)

High bias and high variance
J_{train} will be high
and $J_{cv} \gg J_{train}$

3.5. Regularization and bias/variance

Model: $f_{\vec{w},b}(x) = w_1x + w_2x^2 + w_3x^3 + w_4x^4 + b$

$$J(\vec{w}, b) = \frac{1}{2m}\sum_{i=1}^{m}\left(f_{\vec{w},b}(\vec{x}^{(i)}) - y^{(i)}\right)^2 + \frac{\lambda}{2m}\sum_{j=1}^{n} w_j^2$$



Large λ
High bias (underfit)
$\lambda = 10{,}000$ $w_1 \approx 0, w_2 \approx 0$
$f_{\vec{w},b}(\vec{x}) \approx b$

Intermediate λ

λ

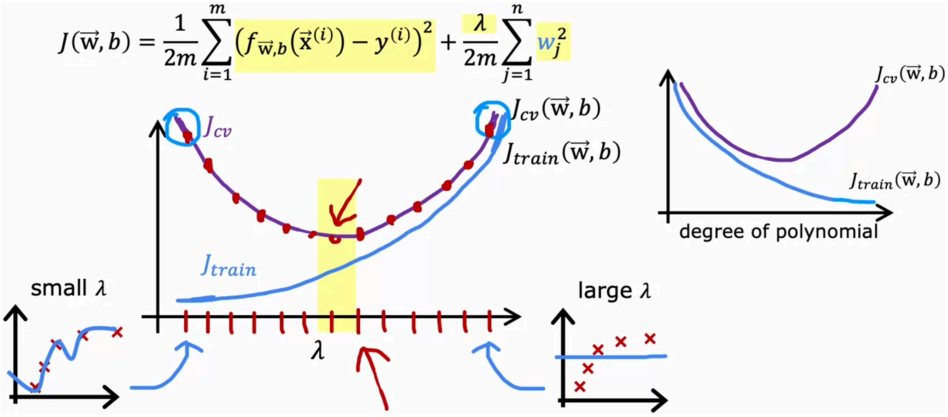
Small λ
High variance (overfit)
$\lambda = 0$

Model: $f_{\vec{w},b}(x) = w_1x + w_2x^2 + w_3x^3 + w_4x^4 + b$

→ 1. Try $\lambda = 0$ → $\min_{\vec{w},b} J(\vec{w}, b)$ → $w^{<1>}, b^{<1>}$ → $J_{cv}(w^{<1>}, b^{<1>})$
→ 2. Try $\lambda = 0.01$ → → $w^{<2>}, b^{<2>}$ → $J_{cv}(w^{<2>}, b^{<2>})$
→ 3. Try $\lambda = 0.02$ → $J_{cv}(w^{<3>}, b^{<3>})$
→ 4. Try $\lambda = 0.04$ $J_{cv}(w^{<5>}, b^{<5>})$
→ 5. Try $\lambda = 0.08$
⋮
→ 12. Try $\lambda \approx 10$ → $w^{<12>}, b^{<12>}$ → $J_{cv}(w^{<12>}, b^{<12>})$

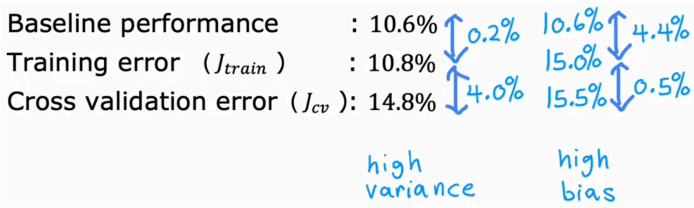
Pick $w^{<5>}, b^{<5>}$

Report test error: $J_{test}(w^{<5>}, b^{<5>})$

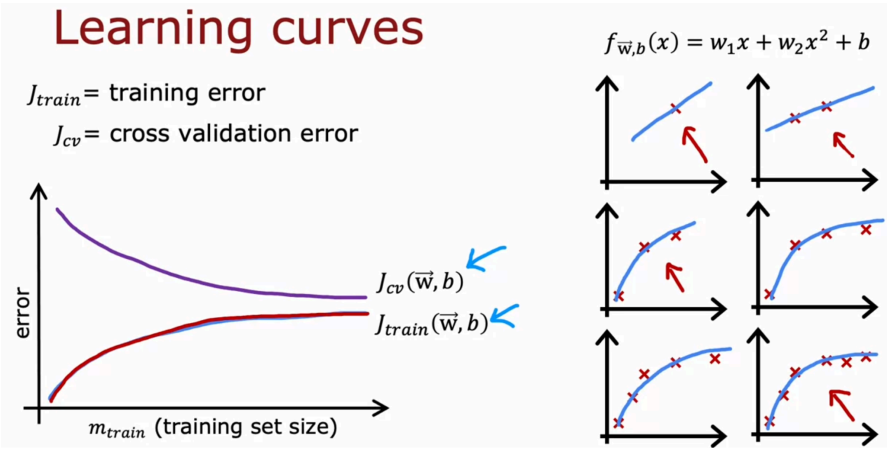


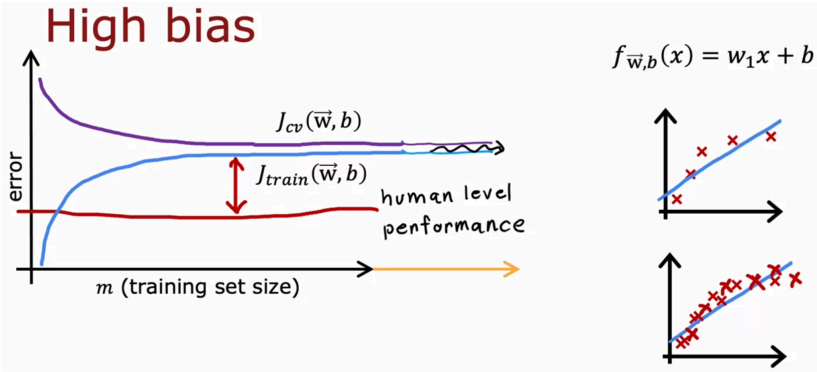
3.6. Establishing a baseline level of performance

- When judging if training error is high, it's useful to establish a baseline performance level: what is the level of error you can reasonably hope to get to?
    - Human level performance
    - Competing algorithms performance
    - Guess based on experience

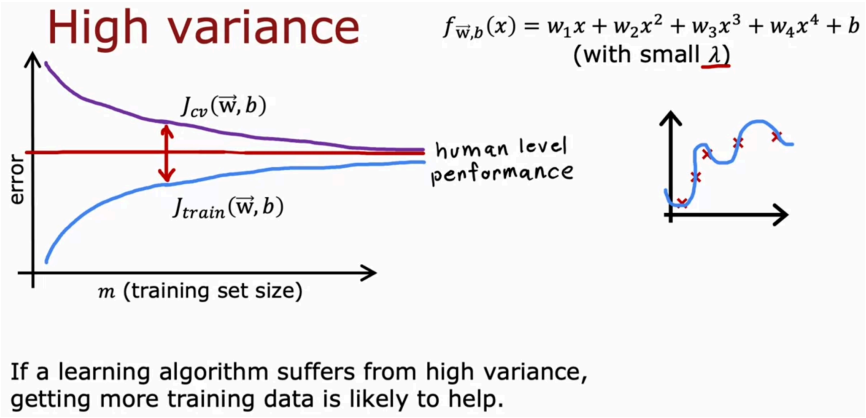


3.7. Learning curves

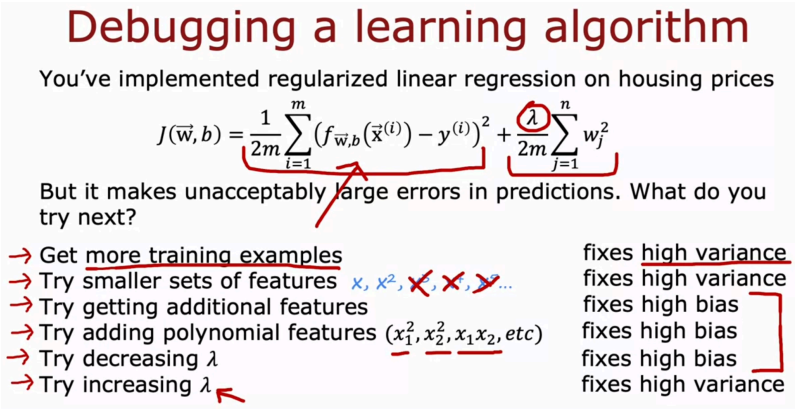




- If a learning algorithm suffers from high bias, getting more training data will not (by itself) help much



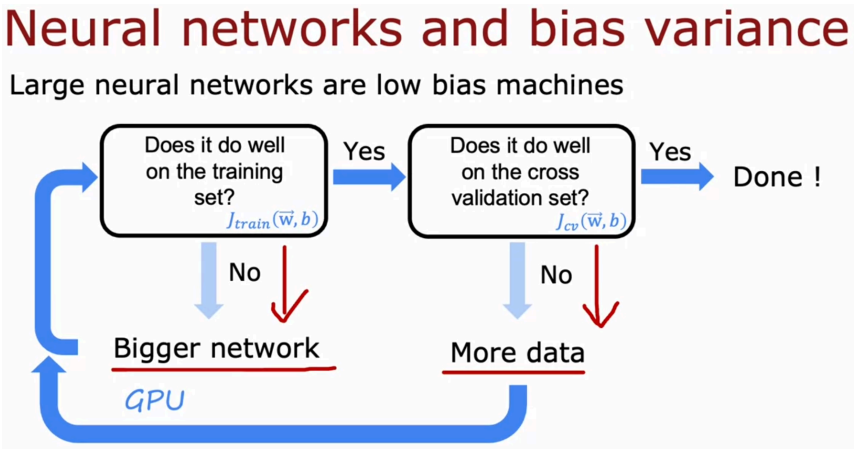
3.8. Deciding what to try next revisited

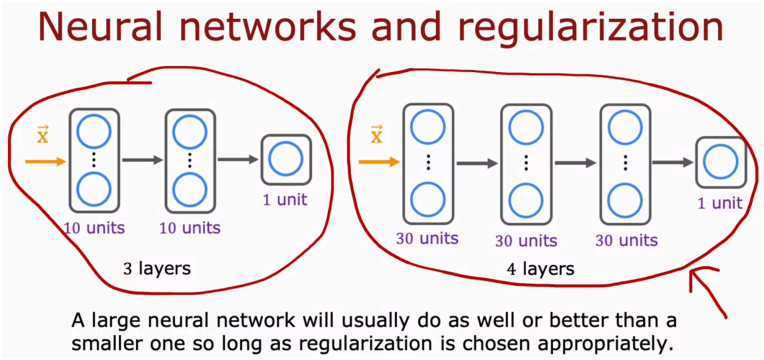


- Can improve high bias by adding complexities to your model (more features, or more complex features) and by decreasing regularization (to allow for more variability)
- Can improve high variance by reducing complexity of your model, increasing regularization, or gathering more training data

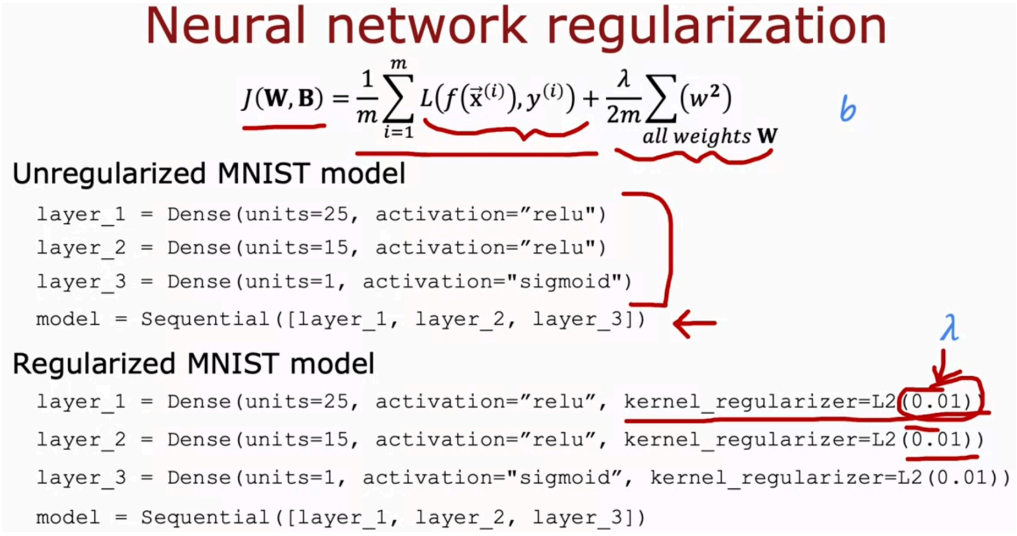
3.9. Bias/variance and neural networks

- Simple model will likely have high bias, but a complex model will have high variance: the bias variance tradeoff
- Large neural networks are low bias machines (because your neural network is usually very large with a lot of parameters)

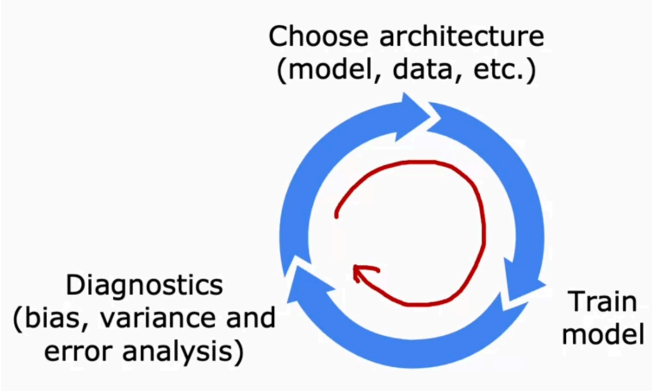




- Larger neural networks will do as well or better than a smaller one, after regularization
- Regularization of a neural network can be done as follows:

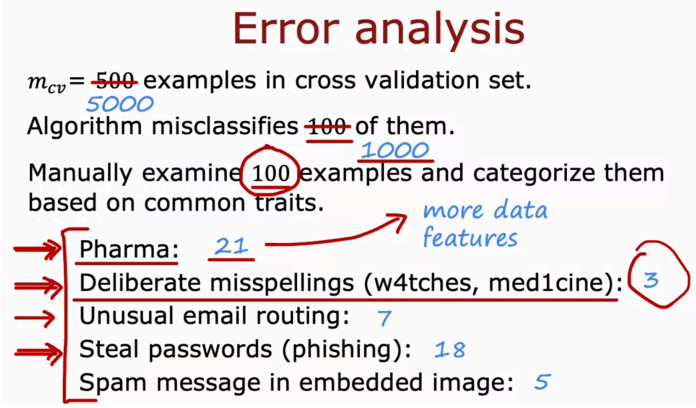


3.10. Iterative loop of ML development



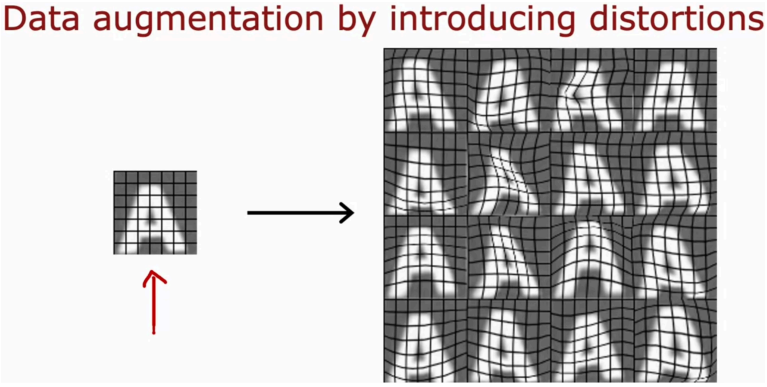
3.11. Error analysis

- Look at errors in cross validation set (misclassifications) and label them manually. Then place efforts on categories with most errors to improve performance.

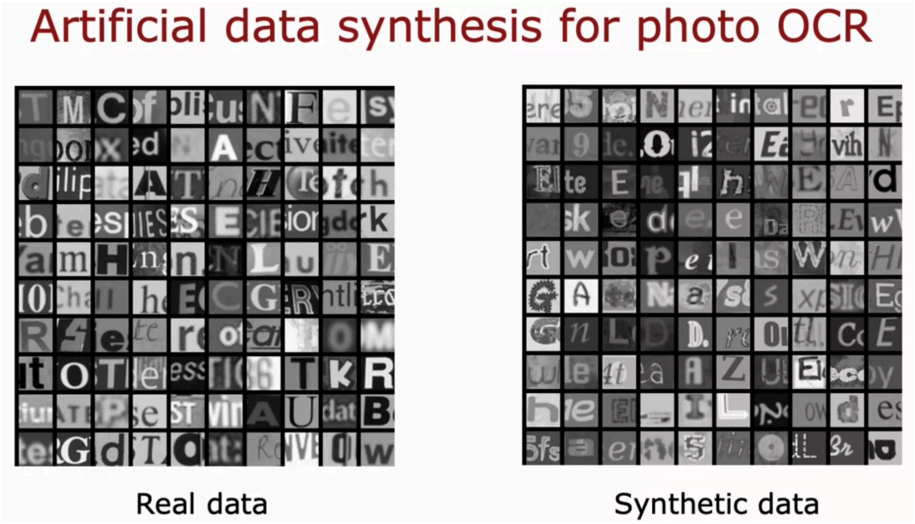


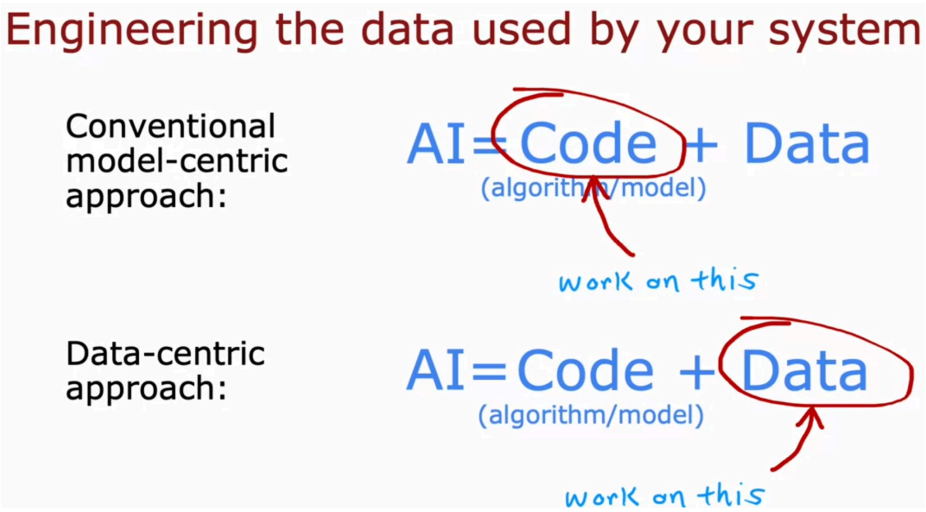
3.12. Adding data

- Trying to get more data on *everything* can be expensive and time consuming. Instead you can try to add more data of the types where error analysis has indicated it might help.
- Beyond getting brand new training examples, another technique is **data augmentation**: modifying an existing training example to create a new training example (rotate, scaling, contrast, flipping, distortion, etc)



- For speech, you can add a noisy background (crowd or car), or add audio on bad cellphone connection for example.
- Usually it does not help to add purely random/meaningless noise to the data. Make the distortions representative of what exists in the testing set.
- **Data synthesis:** using artificial data inputs to create a new training example

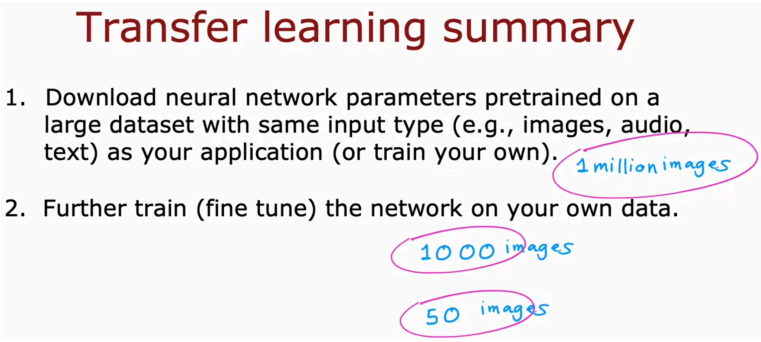




3.13. Transfer learning: using data from a different task

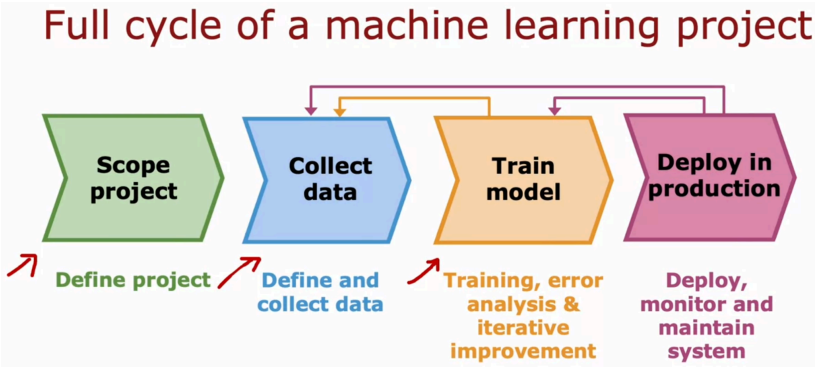
- Using data from a different task
- Train an initial model on a very large readily available dataset and change the final layer to one matching your needs.
- Option 1: only train output layers parameters. Option 2: train all parameters in the network.
- **Supervised pretraining:** train the neural network on a very large dataset for a not-quite related task
- **Fine-tuning:** take the parameters from supervised pretraining and run optimization further to fine-tune the weights to suit the specific application you may have

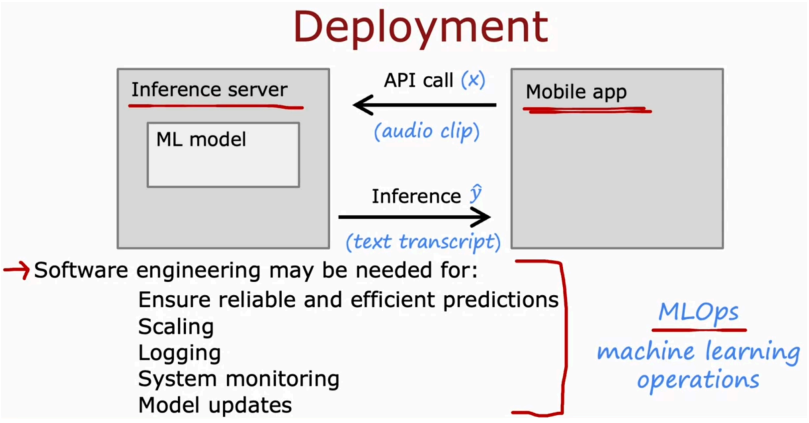
- Why does transfer learning work? Because the shallow (first layers) detect low-level features like edges or corners that are useful for other computer vision tasks.
- Restriction of pre-training is that the input shape has to be the same during pretraining and fine-tuning



3.14. Full cycle of machine learning project

- Scope or define project → define and collect data → training, error analysis, & iterative improvement (sometimes go back and collect more data) → deploy, monitor, & maintain system



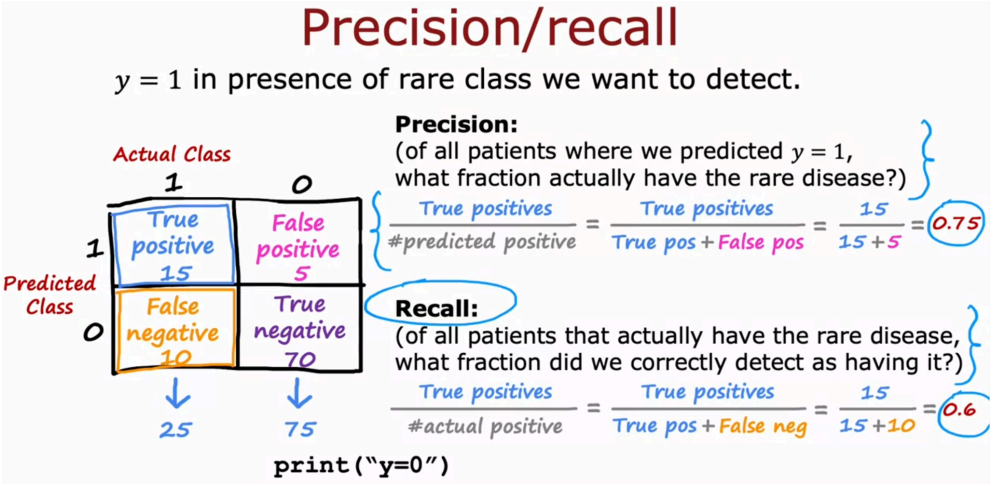


3.15. Fairness, bias, and ethics

- **Bias:** examples include hiring tool that discriminates against women, facial recognition systems matching dark skinned individuals to criminal mugshots, biased bank loan approvals, toxic effect of reinforcing negative stereotypes
- Adverse use cases include deepfakes, spreading toxic/incendiary speech through optimizing for engagement, generating fake content for commercial or political purposes, using ML to build harmful products, commit fraud (spam vs anti spam: fraud vs anti-fraud)
- Guidelines: get a diverse team to brainstorm things that might go wrong, with emphasis on possible harm to vulnerable groups; carry out literature review on standards/guidelines for your industry; audit systems against possible harm prior to deployment; develop mitigation plan (if applicable), and after deployment, monitor for possible harm

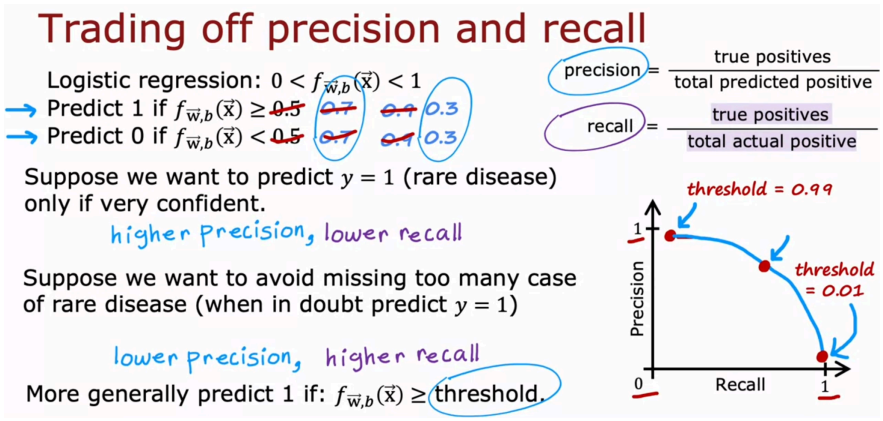
3.16. Error metrics for skewed datasets

- Skewed dataset occurs where the number of positive and negative training examples is far from 50/50.
- Classification error or accuracy is not a good metric for skewed datasets. Instead use precision/recall



3.17. Trading off precision and recall

- If we want to be very confident in our guesses → higher precision, lower recall
- If we want to often find the occurrences → lower precision, higher recall

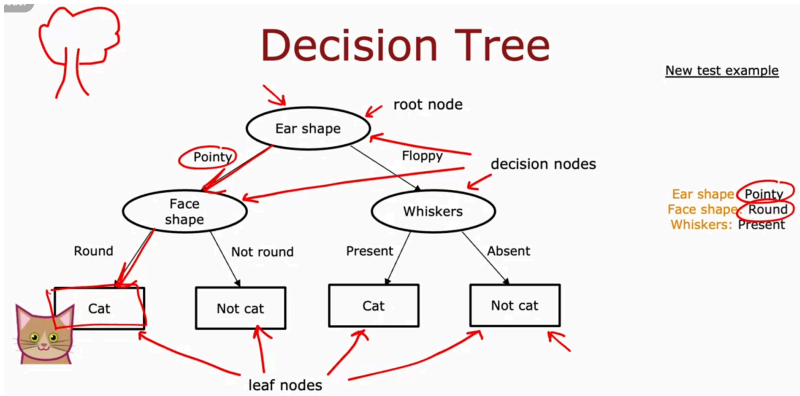


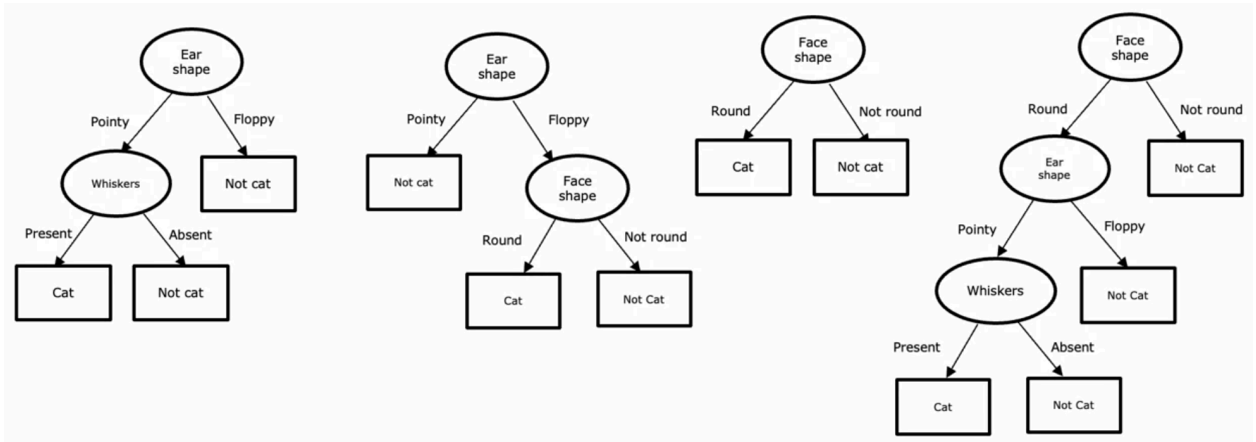
- F1 score is a balanced metric between precision and recall. It is the harmonic mean:
- $F1 = 2 \times \frac{Precision \times Recall}{Precision + Recall}$
- F1 score pays more attention to whichever is lower of precision and recall

4. Week 4 Notes - Decision Trees

4.1. Decision tree model

- Top node of a tree is the root node, the nodes before the last row are called decision nodes, the nodes on the last row are called leaf nodes that make a prediction





4.2. Learning process

- First decide what feature to use at the root node
- Split at each subsequent decision node by selecting different features
- Once all ground truths are the same, you can have a leaf node
- Each node is completely pure if there is no mix of classes
- *Decision 1:* how to choose what features to split on at each node?
    - Maximize purity (or minimize impurity) → get to subsets that are as close as possible to a single class
- *Decision 2:* when do you stop splitting?
    - When a node is 100% one class
    - When splitting a node will result in the tree exceeding a maximum depth
        - Depth is number of hops to get from root node to leaf node
    - When improvements in purity score are below a threshold
    - When number of examples in a node is below a threshold

4.3. Measuring purity

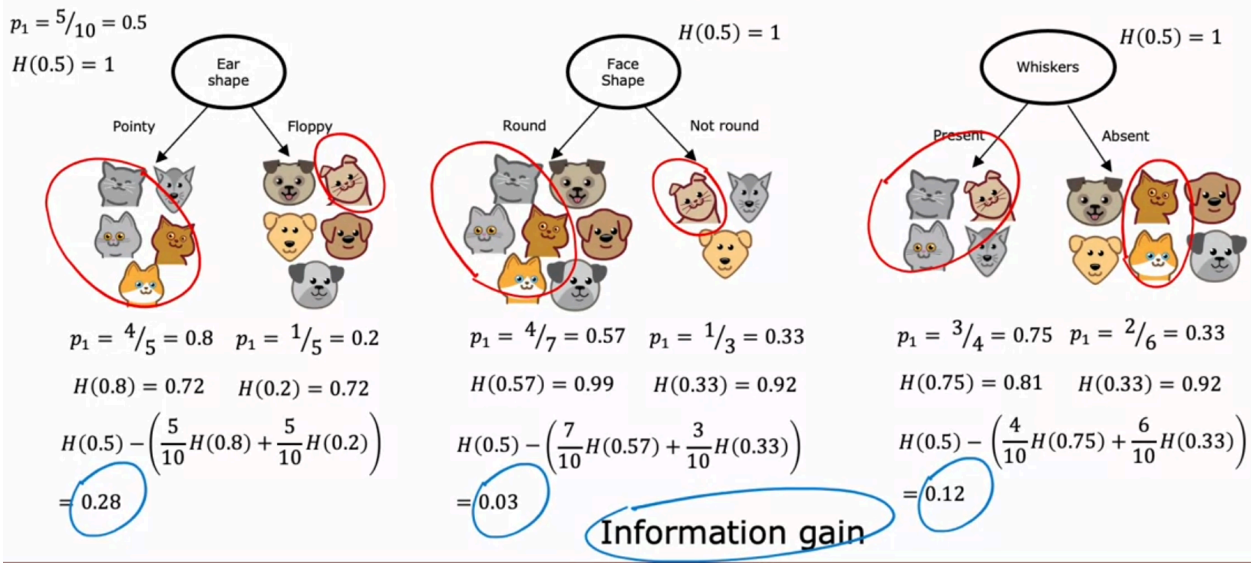
- Let p1 be a fraction of examples that are the positive class
- Entropy function is conventionally denoted as H(p1). Entropy is like impurity

- Most impure at 50/50 split or entropy of 1. Least impurity at 100% or 0% split
- $p_0 = 1 - p_1$
- $H(p_1) = -p_1 log_2(p_1) - p_0 log_2(p_0) = -p_1 log_2(p_1) - (1 - p_1) log_2(1 - p_1)$
- Note "$0log(0)$" $= 0$

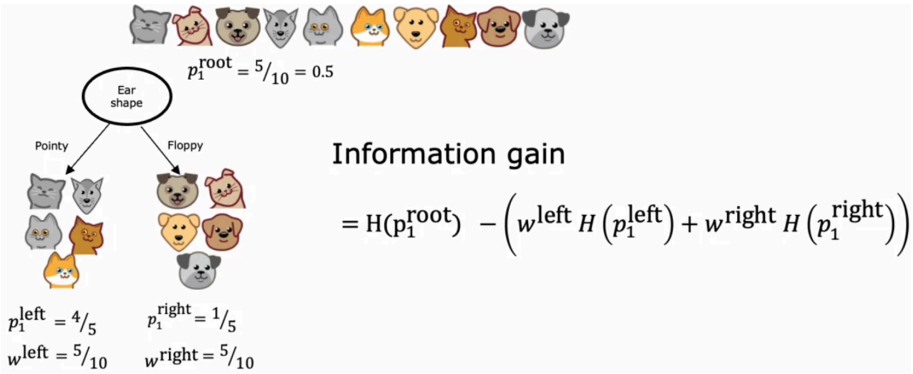
4.4. Choosing a split: Information Gain

- Choose the feature that reduces entropy/impurity the most
- The reduction of entropy is called information gain
- Useful to take a weighted average of left and right node entropies to have a simple value for comparison
- However, to choose a split, you use reduction in entropy: $H(p_{1,root})$ − weighted average

- Choose split with the greatest information gain (or reduction in entropy)



- Can use reduction in entropy as a threshold for deciding when to stop splitting (*i.e.* stop splitting when reduction in entropy is below a certain threshold)



4.5. Putting it together

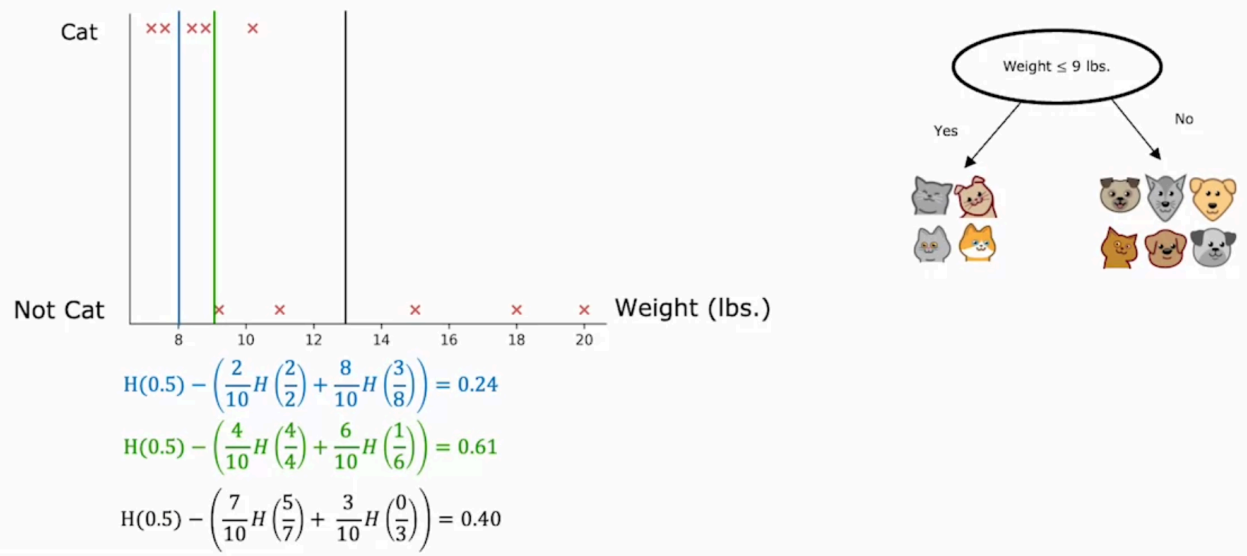
- Start with all examples at the root node
- Calculate information gain for all possible features, and pick the one with highest information gain
- Split dataset according to selected features, and create left and right branches of the tree
- Keep repeating splitting process until stopping criteria is met:
  - When a node is 100% one class
  - When splitting a node will result in the tree exceeding a maximum depth
  - Information gain from additional splits is less than threshold
  - When number of examples in a node is below a threshold
- This is called *recursive splitting*
- You build smaller sub-decision trees from subsets of the data after each step

4.6. Using one-hot encoding of categorical features

- How to address features that have more than 2 discrete values
- Instead of having 1 feature with 3 options, you can split it up into three features with 2 options each (*e.g.,* ear shape: pointy, oval, floppy → pointy ears: 1,0; floppy ears: 1,0; oval ears: 1,0)
- If a categorical feature can take on *k* values, create *k* binary features (0 or 1 valued)

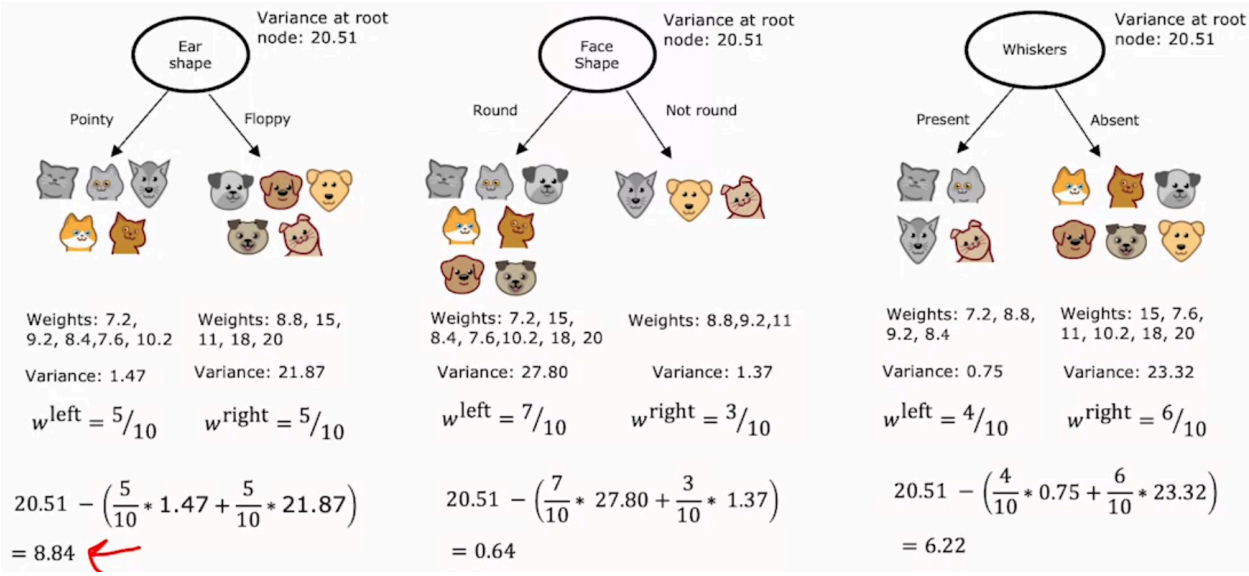
4.7. Continuous valued features

- How to decide how to split on a continuous feature?
- Test different inequality thresholds for the split and calculate information gain at each inequality threshold. Then pick the threshold which leads to the lowest impurity or highest information gain



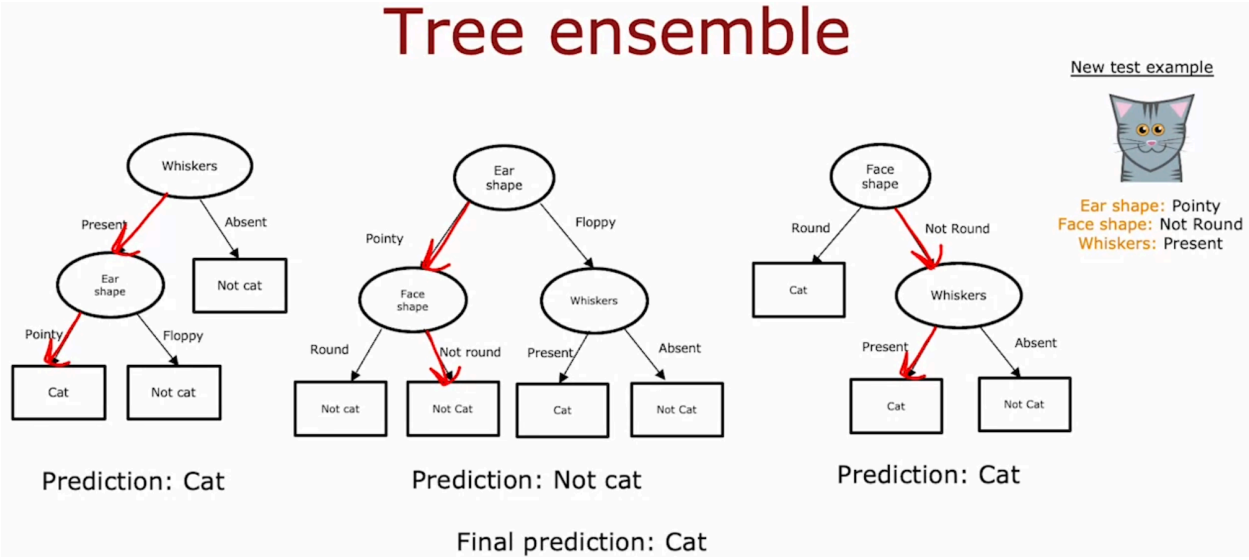
4.8. Regression trees

- Can take the average of examples at each leaf node
- For choosing a split for a regression tree, rather than reducing impurity, we reduce variance of the values, y. Could just chosen option with lowest variance, but, better to use the same information gain formula with variance



4.9. Using multiple decision trees

- A single decision tree can be highly sensitive to small changes in the data
- To increase robustness, build many decision trees → tree ensemble
- Build many trees and run them all on new example, then take the majority vote

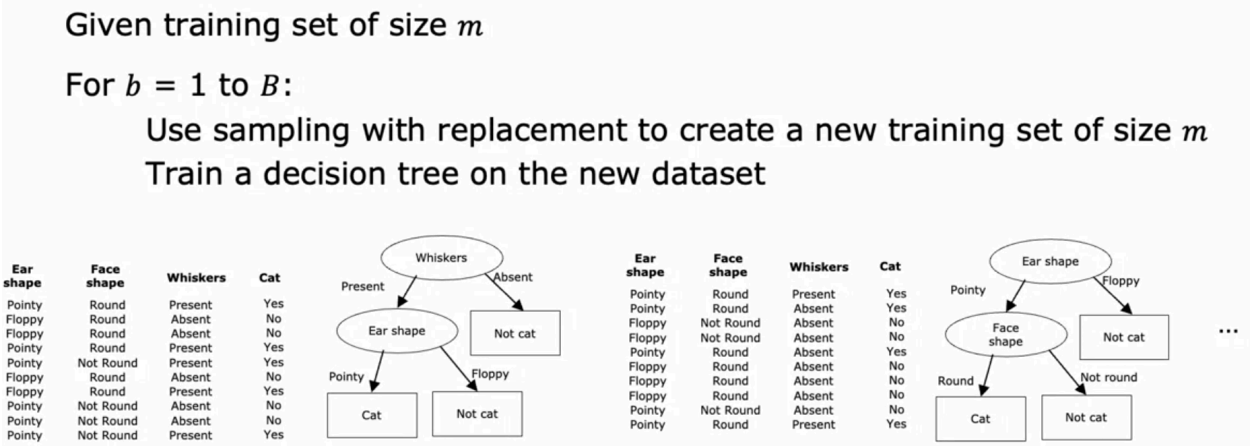


4.10. Sampling with replacement

- With replacement means you will get repeated outcomes even if there is only one instance as you are replacing it before obtaining another instance
- For tree ensemble, we will create random training sets by drawing randomly, with replacement, from a starting master dataset

4.11. Random forest algorithm

- Given a training set of size m
- For $b = 1$ *to* B:
  - Use sampling with replacement to create a new training set of size m
  - Train a decision tree on the new dataset
  - Typical choice of B is usually 64 ~ 128 (larger B never hurts performance, but diminishing returns past a certain point)



- This is called a *bagged decision tree*. This sometimes leads to very similar feature splits in each tree.
- To turn it into a Random forest algorithm, randomize the feature choice:
  - At each node, when choosing a feature to use to split, if n features are available, pick a random subset of $k < n$ features and allow the algorithm to only choose from that subset of features
  - Typical choice is $k = \sqrt{n}$ (for larger problems)

4.12. XGBoost

- Most common use of tree ensembles: quick, effective, many open-source options
- Given a training set of size m
- For $b = 1$ *to* B:
  - Use sampling with replacement to create a new training set of size m
    - But instead of picking from all examples with equal (1/m) probability, make it more likely to pick misclassified examples from previously trained trees → deliberate practice
  - Train a decision tree on the new dataset
- This is called *boosting*
- XGBoost (eXtreme Gradient Boosting):
  - Open-source implementation of boosted trees
  - Fast efficient implementation
  - Good choice of default splitting criteria and criteria for when to stop splitting

- Built-in regularization to prevent overfitting
- Highly competitive algorithm for machine learning competitions
- Rather than sampling with replacement, it assigns weights to each example

Classification

```
from xgboost import XGBClassifier

model = XGBClassifier()

model.fit(X_train, y_train)
y_pred = model.predict(X_test)
```

Regression

```
from xgboost import XGBRegressor

model = XGBRegressor()

model.fit(X_train, y_train)
y_pred = model.predict(X_test)
```

4.13. When to use decision trees

- Decision trees and tree ensembles:
  - Works well on tabular (structured) data
  - Not recommended for unstructured data (images, audio, text)
  - Fast
  - Small decision trees may be human interpretable
- Neural networks:
  - Works well on all types of data, including tabular (structured) and unstructured data as well as mixed data
  - May be slower than a decision tree (takes long to train)
  - Works with transfer learning
  - When building a system of multiple models working together, it might be easier to string together multiple neural networks